FIAP ON

COLHEITA DE SOLUÇÕES INTELIGENTES - TRANSFORMANDO DADOS EM AÇÕES

DESBRAVANDO O DEEP LEARNING: PRIMEIROS PASSOS COM CNN

9

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Estimativa de criação de dados na Internet em 1 minuto.....6
Figura 2 - Tipos de dados e suas proporções.....7
Figura 3 - Relação entre Inteligência Artificial, Machine Learning e Deep Learning .8
Figura 4 - Comparação entre uma Rede Neural Artificial Simples e Uma Rede Neural Profunda.....9
Figura 5 - Diferença entre Machine Learning e Deep Learning.....9
Figura 6 - O impacto de DL em visão computacional, com resultados da ImageNet 11
Figura 7 - Exemplo de aumentação de dados para texto.....13
Figura 8 - O que as CNNs estão aprendendo?.....15
Figura 9 - Componentes típicos de uma CNN.....16
Figura 10 - Comparação de uma RNN (esquerda) com uma rede feedforward (direita).....17
Figura 11 - Comparação de uma RNN com LSTM e GRU.....18
Figura 12 - Representação de uma GAN.....19
Figura 13 - Arquiteturas de Redes Neurais (Profundas ou não).....21
Figura 14 - Representação de uma imagem colorida em RGB.....23
Figura 15 - Representação de uma imagem tons de cinza.....23
Figura 16 - Representação da AlexNet.....24
Figura 17 - Operação de convolução (1).....26
Figura 18 - Operação de convolução (2).....26
Figura 19 - Operação de pooling.....27
Figura 20 - Exemplo de convolução.....27
Figura 21 - Exemplo de pooling.....28
Figura 22 - Exemplo de padding.....29
Figura 23 - Exemplo de striding.....30
Figura 24 - Terminologias de CNNs.....32
Figura 25 - Benchmark de CNNs.....33
Figura 26 - Overview das arquiteturas CNNs até 2018.....34
Figura 27 - Operação de Convolução em Volume.....39
Figura 28 - Exemplos de aumentação de dados.....41
Figura 29 - Exemplos imagens geradas com GANs.....41
Figura 30 - Conceito de Transfer Learning.....42
Figura 31 - Saída da execução do Código-fonte 2.....46
Figura 32 - Saída da execução do Código-fonte 4.....47
Figura 33 - Operação de achatamento de uma imagem.....48
Figura 34 - Saída da execução do Código-fonte 9.....50
Figura 35 - Saída da execução do Código-fonte 10.....51
Figura 36 - Exemplos da base TF_Flowers.....53
Figura 37 - Saída do Código-fonte 14.....54
Figura 38 - Saída do Código-fonte 15.....55
Figura 39 - Saída do Código-fonte 16.....57
Figura 40 - Saída do Código-fonte 17.....58
Figura 41 - Saída do Código-fonte 18.....58
Figura 42 - Saída do Código-fonte 20.....60
Figura 43 - Saída do Código-fonte 21.....61
Figura 44 - Saída do Código-fonte 23.....63

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Funções de ativação para CNNs.....31

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Imports necessários.....45
Código-fonte 2 - Carregando o dataset Fashion MNIST 45
Código-fonte 3 - Explicitando o nome dos labels do dataset.....46
Código-fonte 4 - Exibindo algumas imagens do dataset 46
Código-fonte 5 - Normalizando os dados 47
Código-fonte 6 - Normalizando os dados 48
Código-fonte 7 - Rede neural construída com Sklearn.....49
Código-fonte 8 - Rede neural construída com TensorFlow 49
Código-fonte 9 - Resumindo a rede neural construída com TensorFlow 50
Código-fonte 10 - Treinando a rede neural construída com TensorFlow 51
Código-fonte 11 - Treinando a rede neural construída com TensorFlow 51
Código-fonte 12 - Imports do código de CNN.....53
Código-fonte 13 - Carregando o dataset 54
Código-fonte 14 - Chegando o tamanho dos conjuntos de treino e teste.....54
Código-fonte 15 - Pré-processamento dos dados 55
Código-fonte 16 - Treinamento do modelo baseline, criado a mão 57
Código-fonte 17 - Calculando a acurácia do modelo inicial.....57
Código-fonte 18 - Plotando a matriz de confusão do modelo inicial.....58
Código-fonte 19 - Pré-processamento dos dados para uso da VGG16 para Transfer Learning 59
Código-fonte 20 - Observando as camadas da VGG16 59
Código-fonte 21 - Completando o modelo de transfer com camadas de classificação 61
Código-fonte 22 - Compilando o treinando o modelo resultante 62
Código-fonte 23 - Avaliação do modelo com Transfer Learning.....62

FIXED

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 DESBRAVANDO O DEEP LEARNING: PRIMEIROS PASSOS COM CNN 6
1.1 Precisamos de novos algoritmos de Aprendizado de Máquina? 6
1.2 Conceitos e Histórico de Deep Learning 8
1.3 E a trajetória do Deep Learning?.....10
1.4 Viabilizadores e aplicabilidade 12

2 FRAMEWORKS DE DEEP LEARNING 13

3 PRINCIPAIS ARQUITETURAS 15
3.1 Redes Neurais Convolucionais 15
3.2 Redes Neurais Recorrentes 16
3.3 Redes Adversárias Generativas.....19
3.4 Outras arquiteturas.....20

4 ARQUITETURAS CONVOLUCIONAIS 22
4.1 Representação de uma imagem 22
4.2 Estrutura básica 24
4.3 Entendendo operações de Convolução e Pooling.....25
4.4 Configurações avançadas 29
4.4.1 Padding e Striding 29
4.4.2 Funções de ativação 30
4.5 Terminologia e evolução 32

5 TREINAMENTO DE REDES NEURAIS CONVOLUCIONAIS.....36
5.1 Preparação de dados 36
5.2 Técnicas de regularização.....37
5.3 Otimização de hiperparâmetros 37
5.4 Operações sobre volumes.....38

6 LIDANDO COM DESAFIOS EM REDES NEURAIS CONVOLUCIONAIS 40
6.1 Técnicas de ampliação da base de dados 40
6.2 Transfer Learning e Fine-tuning 42

7 HANDS ON: REDES NEURAIS PROFUNDAS.....44
7.1 Deep Learning em Python.....44
7.2 CNN em Python 52

REFERÊNCIAS.....64

GLOSSÁRIO 67

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 DESBRAVANDO O DEEP LEARNING: PRIMEIROS PASSOS COM CNN

1.1 Precisamos de novos algoritmos de Aprendizado de Máquina?

No mundo moderno, somos banhados por um oceano de dados. A cada segundo, uma quantidade exorbitante de informações é gerada, capturada e compartilhada em todas as esferas da nossa sociedade digitalizada. Desde transações comerciais até interações em redes sociais, passando por registros médicos e dispositivos de IoT, os dados fluem incessantemente, inundando nossos sistemas de armazenamento e desafiando nossa capacidade de compreensão.

Para ilustrar a produção de dados na era da Internet, a figura “Estimativa de criação de dados na Internet em 1 minuto” mostra a quantidade estimada de dados em 1 minuto, de acordo com o aplicativo/plataforma, para o ano de 2021. Imagine como estes números aumentaram até hoje!

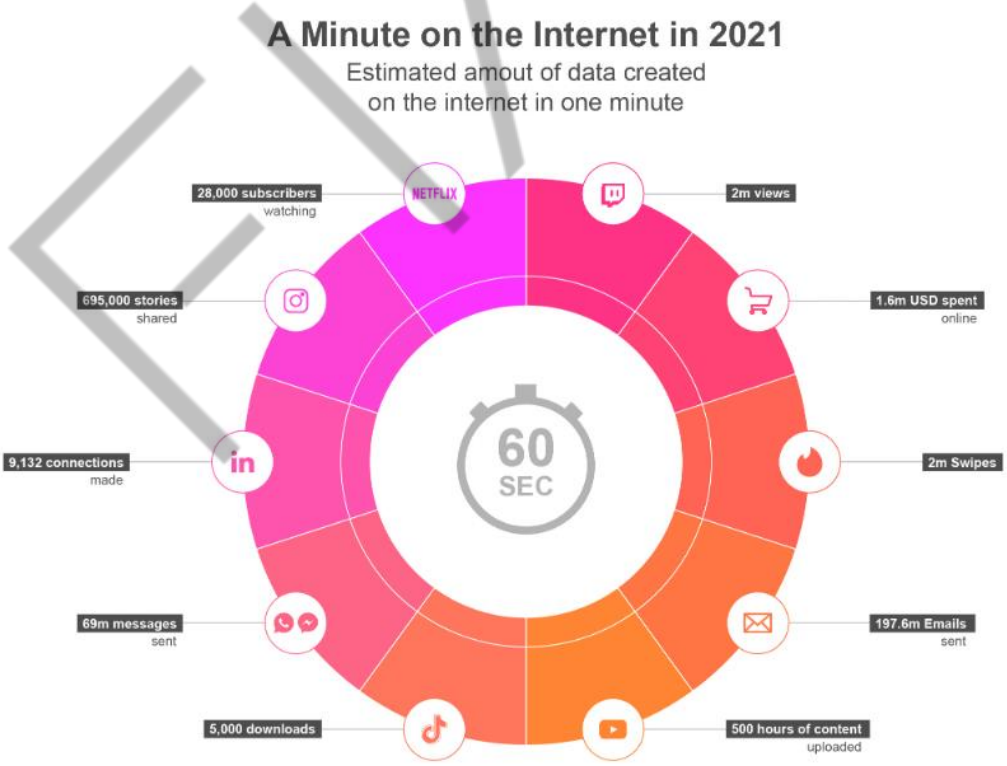

Figura 1 - Estimativa de criação de dados na Internet em 1 minuto
Fonte: Statista (2021), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Entretanto, a maior parte dos dados gerados e compartilhados é complexa e não estruturada. Estima-se que cerca de 80% de todos os dados produzidos sejam não estruturados, representando imagens, vídeos, áudios, textos, e-mails, postagens em redes sociais, entre tantas outras formas e fontes. A proporção e fontes de dados estão ilustrados na figura a seguir.

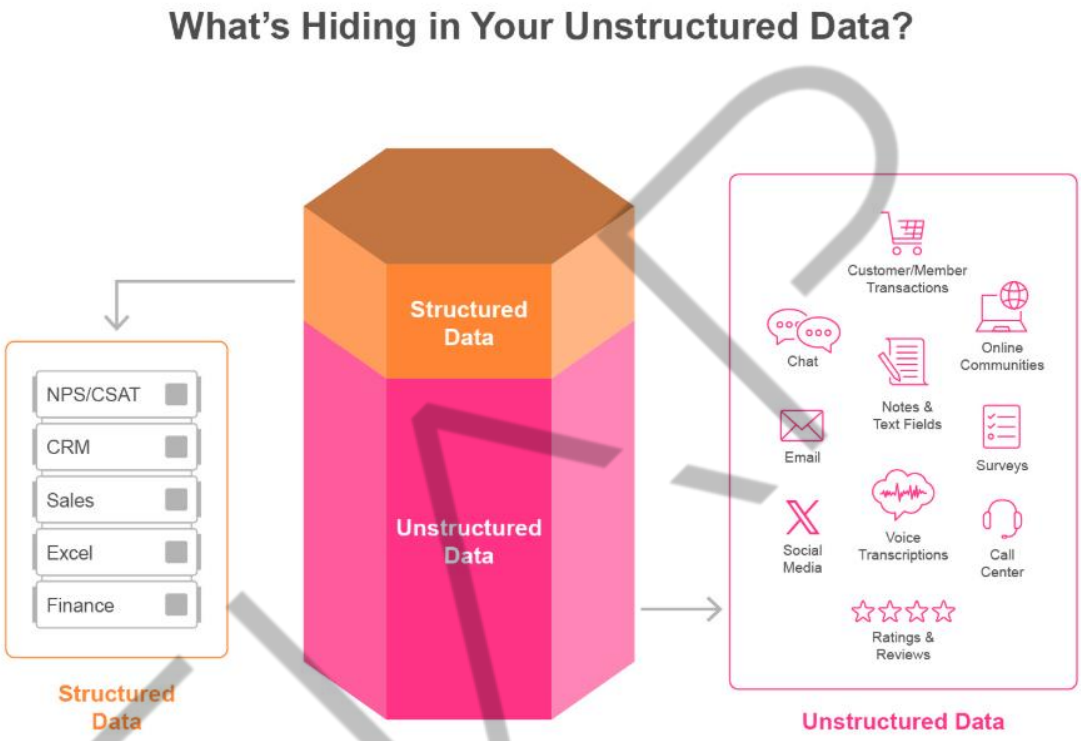

Figura 2 - Tipos de dados e suas proporções
Fonte: Jelvix (2024), adaptado por FIAP (2025)

Toda essa complexidade e robustez resiste às técnicas tradicionais de processamento e análise de dados, exigindo abordagens mais sofisticadas para extrair informações significativas. Nesse contexto, surge uma pergunta crucial: **como lidar com essa avalanche de dados e transformá-la em conhecimento acionável?**

É nesse ponto que surge a necessidade de novos algoritmos de Aprendizado de Máquina, pois os métodos tradicionais podem ser eficazes para lidar com dados estruturados e tarefas relativamente simples, porém, muitas vezes sucumbem diante da complexidade e da escala dos dados não estruturados. Para desvendar os insights ocultos nessas montanhas de dados, precisamos de ferramentas mais poderosas, capazes de aprender padrões sutis e extrair informações valiosas de maneira eficiente

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

e escalável. É aqui que o **Deep Learning** (DL) entra em cena, oferecendo promessas de descobertas revolucionárias em um mundo inundado de dados.

1.2 Conceitos e Histórico de Deep Learning

O **Deep Learning** (Aprendizado Profundo) é uma subárea do campo de **Inteligência Artificial** (IA) que se concentra no treinamento de modelos de **Machine Learning** (Aprendizado de Máquina) em conjuntos de dados muito grandes e complexos. Nesta etapa, é importante relembrar a relação entre os conceitos, reforçado na figura “Relação entre Inteligência Artificial, Machine Learning e Deep Learning”. É devida a esta relação que o treinamento de modelos de DL segue os mesmos passos e estratégias de Machine Learning (ML), por exemplo, separação entre treino e teste, etapas de validação, cálculo de métricas sob o conjunto de teste, entre outros.

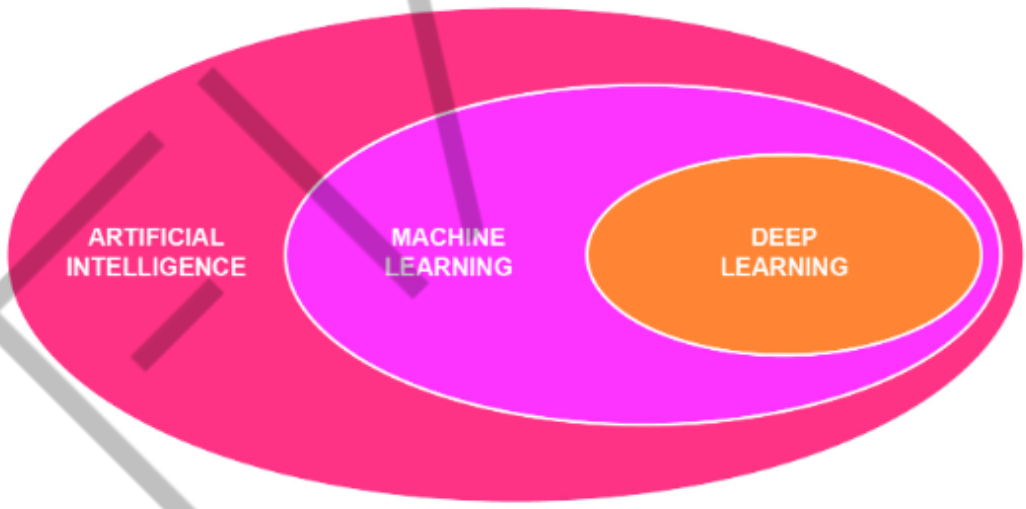

Figura 3 - Relação entre Inteligência Artificial, Machine Learning e Deep Learning
Fonte: Medium (2020), adaptado por FIAP (2025)

Os modelos de Aprendizado de Máquina que servem de base para o Deep Learning são chamados de **Redes Neurais Artificiais** (RNAs). A principal diferença entre o DL e as RNAs, é que o primeiro emprega **diversas** camadas ocultas para aprender representações e correlações dos dados. Observe a figura:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

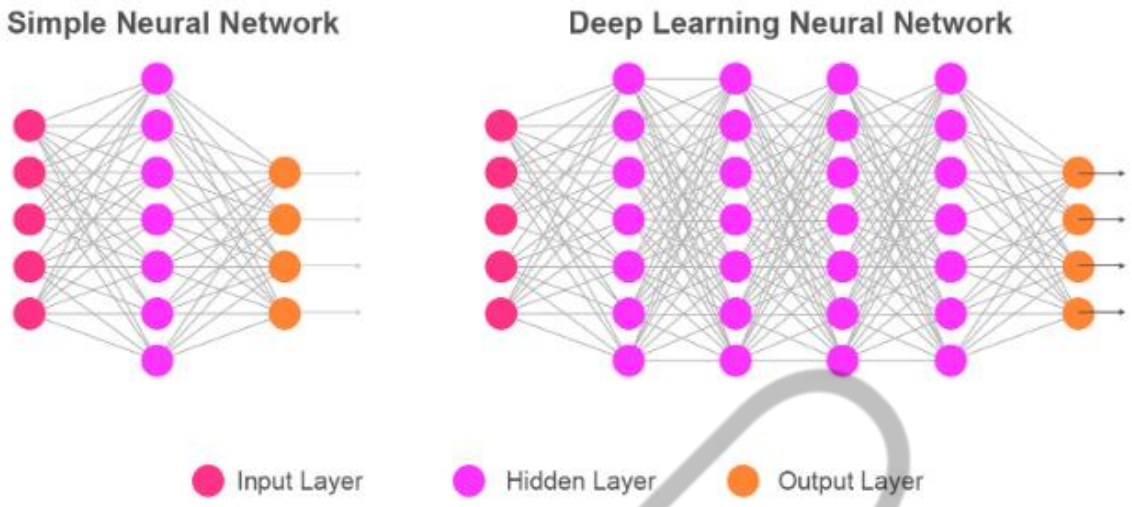

Figura 4 - Comparação entre uma Rede Neural Artificial Simples e Uma Rede Neural Profunda
Fonte: The Data Scientist (2018), adaptado por FIAP (2025)

Ao contrário de métodos tradicionais de Aprendizado de Máquina, que dependem de características cuidadosamente projetadas e extraídas manualmente dos dados, o **Deep Learning aprende automaticamente representações hierárquicas a partir de dados brutos**. Observe a figura:

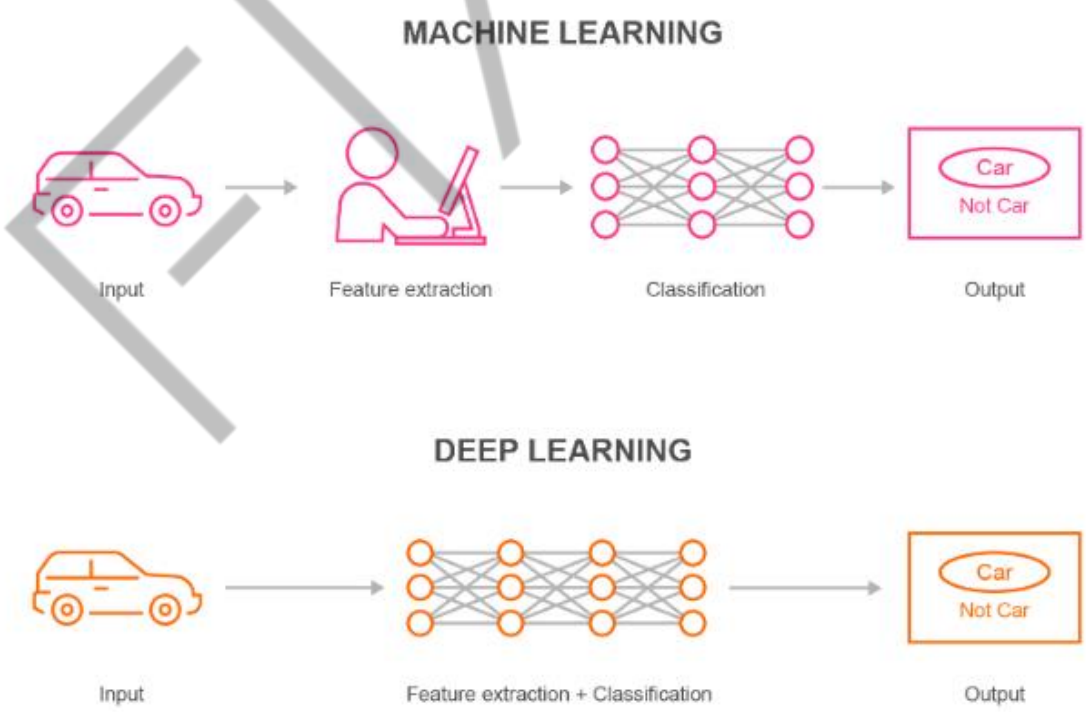

Figura 5 - Diferença entre Machine Learning e Deep Learning
Fonte: Medium (2018), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Essas representações hierárquicas permitem que os modelos aprendam características cada vez mais abstratas e complexas dos dados, possibilitando a resolução de tarefas extremamente complexas, como reconhecimento de imagens, processamento de linguagem natural e previsão em uma variedade de domínios.

Mas **se engana quem pensa que o Deep Learning dispensa as etapas de limpeza e preparação de dados**. Embora os modelos de Deep Learning sejam capazes de aprender características complexas diretamente dos dados brutos, é crucial garantir que os dados de entrada estejam limpos, organizados e preparados adequadamente para garantir o desempenho ideal do modelo. A qualidade dos dados tem um impacto significativo na eficácia e na precisão dos modelos de Deep Learning. Portanto, a limpeza e a preparação de dados continuam sendo etapas essenciais no processo de desenvolvimento de modelos de Deep Learning.

De modo geral, o Deep Learning viabiliza o processamento e a interpretação de dados armazenados sob tipos complexos, com pouca intervenção humana na criação de features. Em contrapartida, os modelos de DL são ávidos por dados e exigem alto poder computacional para treinamento. Ainda assim, seu crescimento e resultados promissores em análises preditivas ou tomada de decisões abrem portas para uma variedade de aplicações inovadoras em diversas indústrias.

1.3 E a trajetória do Deep Learning?

A história do Deep Learning é uma evolução da história das RNAs. Até 2011, as RNAs eram mais uma abordagem de Machine Learning, que podiam ou não entregar bons resultados em tarefas de regressão ou classificação.

O marco significativo ocorreu em 2012, na competição ImageNet. A competição ImageNet, ou **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)**, é uma competição anual que visa avançar o estado da arte em visão computacional e reconhecimento de padrões. O desafio é classificar objetos presentes em imagens em milhares de categorias, utilizando um conjunto de dados extremamente grande e diversificado chamado ImageNet. Esse conjunto possui mais de 14 milhões de imagens divididas em mais de 20 mil categorias.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nos anos anteriores da competição (2010 e 2011), as performances dos modelos eram modestas (com taxas de ERRO em torno de 28%), mas em 2012 a AlexNet, uma rede neural profunda, obteve um desempenho impressionante, superando os métodos tradicionais por uma margem significativa. Observe a figura:

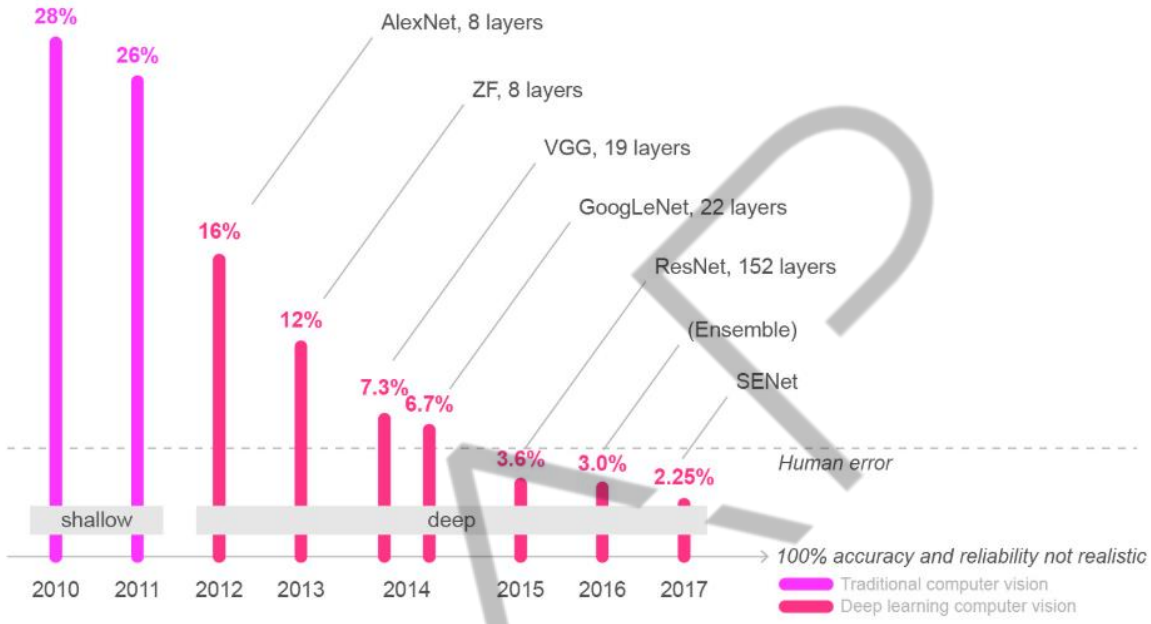

Figura 6 - O impacto de DL em visão computacional, com resultados da ImageNet
Fonte: Semiconductor Engineering (2016), adaptado por FIAP (2025)

A AlexNet reduziu o erro de classificação em mais de 10% em relação ao primeiro colocado do ano anterior, demonstrando a capacidade do Deep Learning em lidar com conjuntos de dados complexos e de grande escala.

A figura “O impacto de DL em visão computacional, com resultados da ImageNet” mostra, ainda, que a partir de 2012 todos os outros ganhadores da ImageNet propuseram modelos baseados em DL também, cujas taxas de erro superam a capacidade humana de reconhecimento dos objetos das imagens da base. **É curioso notar que uma rede “profunda” em 2012 possuía apenas oito camadas! Em contraste, o melhor modelo de 2015 possuía 152 camadas!**

A partir do marco introduzido em 2012 pela AlexNet, houve um aumento exponencial na pesquisa e no desenvolvimento de técnicas de Deep Learning, levando a avanços significativos em várias áreas, como visão computacional, processamento de linguagem natural e reconhecimento de voz, nos anos seguintes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.4 Viabilizadores e aplicabilidade

A ascensão do Deep Learning no início da década de 2010 foi impulsionada por três vários fatores-chave:

1. **Avanços computacionais:** permitindo o treinamento eficiente de modelos complexos em grandes conjuntos de dados;
2. **Avanços em dados:** acesso a conjuntos de dados massivos e rotulados, como o ImageNet, proporcionou a oportunidade de treinar modelos de Deep Learning em tarefas de reconhecimento de imagens em escala sem precedentes;
3. **Avanços de algoritmos:** como Redes Neurais Convolucionais (CNNs) e Redes Neurais Recorrentes (RNNs), que mostraram um desempenho superior em uma variedade de tarefas de inteligência artificial.

O Deep Learning tem transformado diversas áreas da tecnologia e ciência, impulsionando avanços significativos em Inteligência Artificial. No campo da visão computacional, Redes Neurais Convolucionais (CNNs) são utilizadas em reconhecimento facial, diagnóstico médico por imagem e veículos autônomos. No Processamento de Linguagem Natural (PLN), modelos como transformers possibilitam traduções automáticas, assistentes virtuais e geração de texto. Além disso, o Deep Learning é aplicado em sistemas de recomendação, personalizando conteúdos em plataformas de streaming e e-commerce. No setor financeiro, ajuda na detecção de fraudes e análise de riscos, enquanto na biotecnologia acelera a descoberta de medicamentos. Seu impacto continua a crescer, tornando-se um pilar essencial da inovação tecnológica.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 FRAMEWORKS DE DEEP LEARNING

Assim como existem bibliotecas específicas para trabalhar com Machine Learning, como o scikit-learn e o SparkML, também existem diversos frameworks para Deep Learning, essenciais para desenvolver, treinar e implantar modelos de aprendizado profundo de maneira eficiente.

Os frameworks fornecem uma série de recursos e abstrações que simplificam o processo de construção de modelos complexos e permitem a experimentação rápida com diferentes arquiteturas e técnicas. Existem diversos frameworks disponíveis, cada um com suas características e vantagens específicas. Além disso, esses frameworks estão em constante evolução, incorporando novos recursos e otimizações para atender às crescentes demandas da comunidade de aprendizado profundo.

A figura a seguir mostra o estado da arte dos frameworks de DL em 2017, e que, desde então, a evolução foi imensa. Atualmente, entre os principais frameworks de DL, destacam-se **TensorFlow**, **PyTorch**, **Keras**, **Caffe** e **MXNet**.

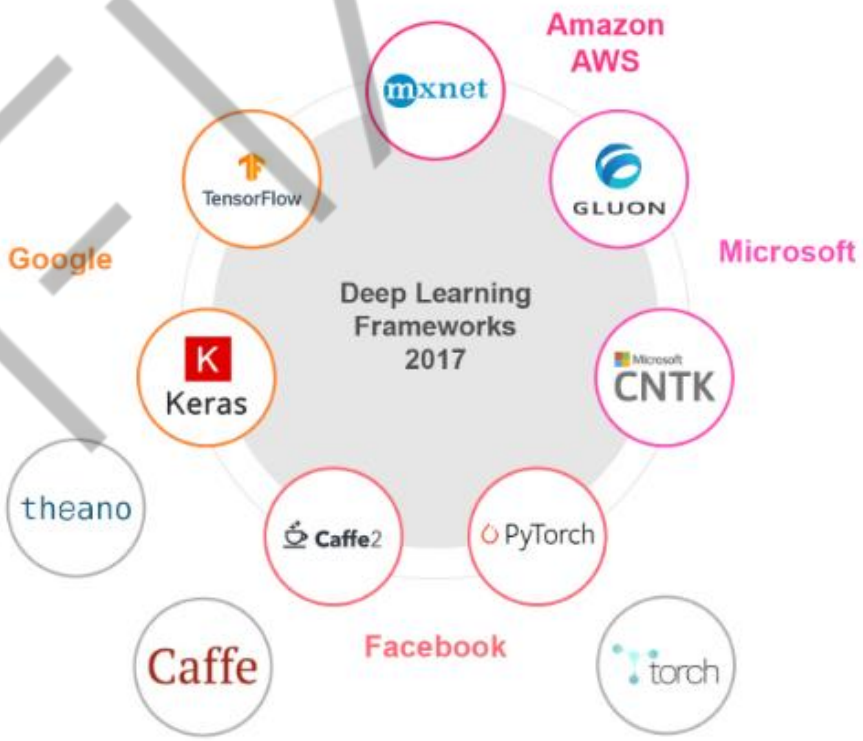

Figura 7 - Exemplo de aumentação de dados para texto
Fonte: Medium (2017), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Entre os principais frameworks, o **TensorFlow**, desenvolvido pelo Google, é um dos mais populares e amplamente utilizado, oferecendo uma ampla gama de ferramentas para desenvolvimento de modelos, desde redes neurais convolucionais até modelos recorrentes. O **PyTorch**, desenvolvido pelo Facebook, é conhecido por sua flexibilidade e simplicidade, sendo amplamente adotado pela comunidade de pesquisa. Já o **Keras** é uma API de alto nível que pode ser executada em cima de outros frameworks, como **TensorFlow** e **Theano**, facilitando a construção rápida de protótipos e experimentos.

Desenvolvido em C++ com interface para Python, o **Caffe** é conhecido por sua velocidade e eficiência, e era frequentemente utilizado em aplicações de visão computacional. Sua evolução, o **Caffe2**, foi introduzida em 2017 e em 2018, porém, foi incorporado ao PyTorch. Por fim, o **MXNet** (mantido pela Apache) é conhecido por sua escalabilidade e eficiência em ambientes distribuídos, mas seu desenvolvimento ativo se encerrou em 2023.

A importância desses frameworks reside no fato de que eles permitem que os desenvolvedores construam e treinem modelos complexos de Deep Learning de maneira mais rápida e eficiente, sem a necessidade de implementar manualmente todos os detalhes da rede neural (imagine então de uma rede neural **profunda**!). Eles fornecem APIs intuitivas, uma variedade de módulos pré-implementados e suporte para aceleração de hardware, como GPUs, o que é essencial para lidar com conjuntos de dados grandes e complexos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 PRINCIPAIS ARQUITETURAS

Existem diversas arquiteturas de Deep Learning. A seguir será apresentado as principais arquiteturas. Vale mencionar que, neste momento, não temos a intenção de sermos detalhistas, mas apenas diferenciar os modelos, destacando suas motivações, particularidades e aplicações.

3.1 Redes Neurais Convolucionais

As Redes Neurais Convolucionais (do Inglês “Convolutional Neural Network”, CNN) são um tipo especial de rede neural projetada para processar e analisar dados de forma semelhante ao cérebro humano, especialmente quando se trata de imagens e vídeos. Elas são chamadas de “convolucionais” porque utilizam uma operação matemática chamada convolução para processar os dados de entrada.

Essas redes são compostas por várias camadas, cada uma com um conjunto específico de funções. Na camada inicial, a rede recebe a imagem como entrada e aplica filtros para extrair características simples, como bordas e cores. À medida que avança para camadas mais profundas, as características extraídas tornam-se mais complexas e abstratas, permitindo à rede reconhecer padrões cada vez mais sofisticados, como formas, texturas e objetos. Observe a figura:

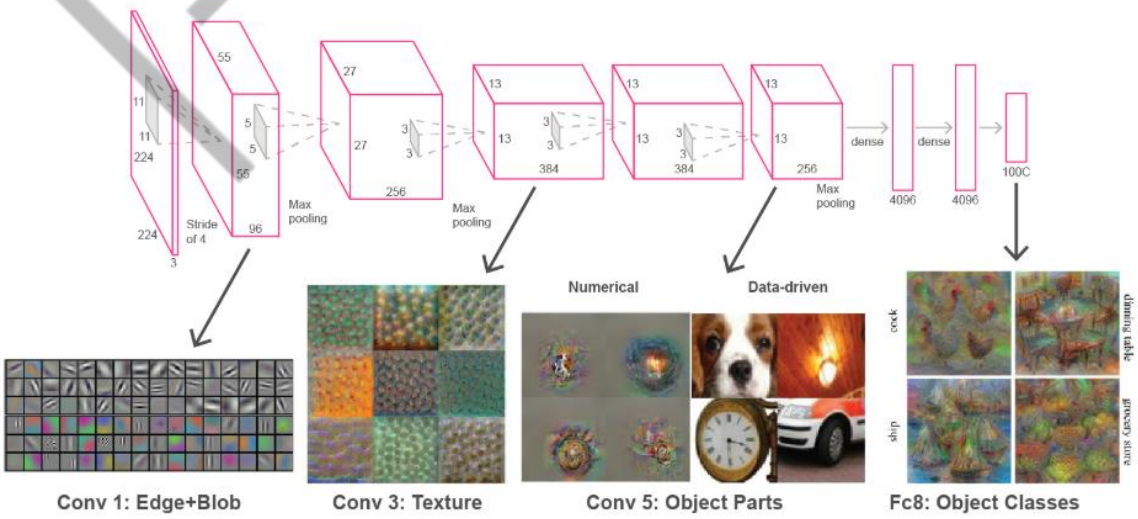

Figura 8 - O que as CNNs estão aprendendo?
Fonte: GitHub [2020], adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Uma das características mais importantes das redes neurais convolucionais é a capacidade de compartilhar pesos. Isso significa que os mesmos filtros são aplicados em diferentes partes da imagem, o que reduz significativamente o número de parâmetros a serem aprendidos e torna o modelo mais eficiente. Além disso, as camadas convolucionais são frequentemente seguidas por camadas de pooling, que reduzem a dimensão dos dados, preservando as características mais importantes.

Observe a figura:

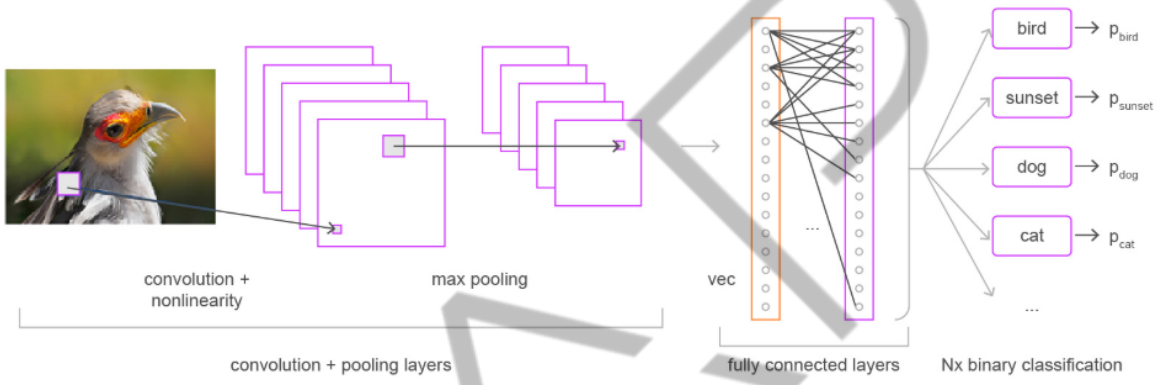

Figura 9 - Componentes típicos de uma CNN
Fonte: GitHub (2016), adaptado por FIAP (2025)

Essa combinação de convolução e pooling permite que as redes neurais convolucionais aprendam representações hierárquicas dos dados, o que é fundamental para seu sucesso em tarefas de reconhecimento de padrões, como classificação de imagens e detecção de objetos.

3.2 Redes Neurais Recorrentes

As Redes Neurais Recorrentes (do Inglês “Recurrent Neural Network”, RNNs) são um tipo especial de rede neural projetado para lidar com dados sequenciais, como séries temporais, texto e áudio. Ao contrário das redes neurais convencionais, que processam cada entrada de forma independente, as RNNs possuem loops internos que lhes permitem manter informações sobre o estado anterior enquanto processam a entrada atual. Isso as tornam capazes de capturar dependências temporais e contextuais nos dados, tornando-as ideais para tarefas que envolvem sequências de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A principal diferença entre uma RNN e uma rede padrão (feedforward) está ilustrada na figura “Comparação de uma RNN (esquerda) com uma rede feedforward (direita)”, que são as arestas representando os loops internos de cada neurônio, os quais simbolizam a memorização da informação:

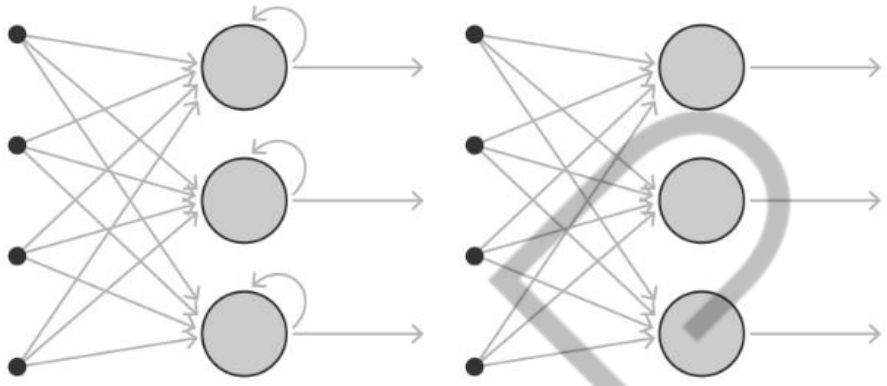

Figura 10 - Comparação de uma RNN (esquerda) com uma rede feedforward (direita)
Fonte: MDPI (2022), adaptado por FIAP (2025)

A estrutura em loop das RNNs permite que elas processem entradas de comprimento variável e produzam saídas de acordo com o contexto da entrada. Cada passo de tempo em uma RNN recebe uma entrada e uma representação do estado oculto anterior como entrada, e produz uma saída e uma nova representação do estado oculto. Essa capacidade de manter uma memória de longo prazo e atualizá-la dinamicamente ao longo do tempo é fundamental para tarefas como tradução automática, geração de texto, reconhecimento de fala e previsão de séries temporais.

No entanto, as RNNs tradicionais possuem algumas limitações, como dificuldade em capturar dependências de longo prazo e propensão ao desaparecimento do gradiente. Para lidar com esses problemas, surgiram variações mais avançadas das RNNs, destacando-se:

- As **Redes Neurais Recorrentes de Memória de Curto Longo Prazo** (do Inglês “Long Short-Term Memory”, LSTMs);
- As **Unidades Recorrentes Fechadas** (do Inglês “Gated Recurrent Units”, GRUs).

Ambas as arquiteturas são capazes de aprender e reter informações ao longo de sequências muito longas, tornando-as mais eficazes em uma variedade de tarefas

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de Aprendizado de Máquina. A principal diferença entre as LSTMs e as GRUs está na sua complexidade e na quantidade de portões de controle que possuem. Sem entrar em muitos detalhes, as LSTMs possuem três portões de controle: o **portão de esquecimento**, o **portão de entrada** e o **portão de saída**, enquanto as GRUs possuem apenas dois portões: o **portão de atualização** e o **portão de reset**. Simplificando, estes portões são responsáveis por estabelecer qual informação continuará retida na rede e qual deve ser esquecida.

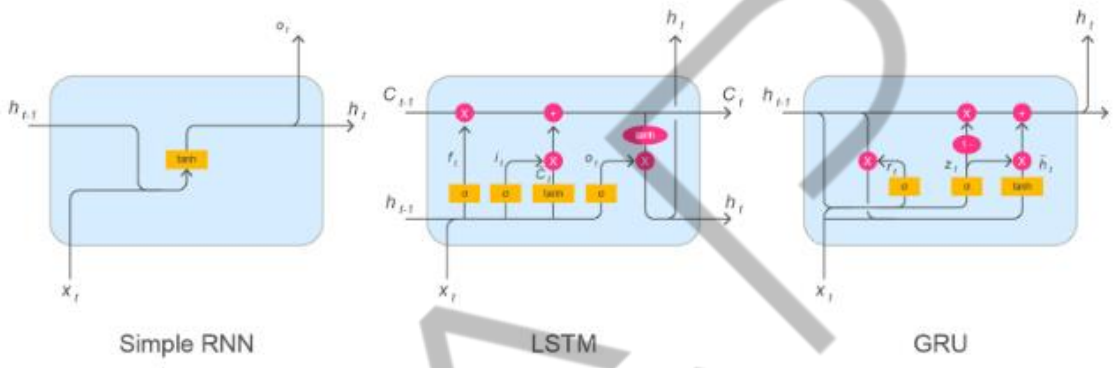

Figura 11 - Comparação de uma RNN com LSTM e GRU
Fonte: wikidocs.net (2025), adaptado por FIAP (2025)

As LSTMs são mais complexas e têm mais parâmetros, o que as tornam capazes de aprender relações temporais mais longas. No entanto, essa complexidade também pode tornar o treinamento mais lento e propenso a overfitting em conjuntos de dados menores. Por outro lado, as GRUs são mais simples e têm menos parâmetros, o que as tornam mais rápidas para treinar e menos propensas a overfitting. No entanto, essa simplicidade também pode limitar sua capacidade de capturar dependências de longo prazo em alguns casos.

As RNNs, juntamente com suas variantes LSTM e GRU, são amplamente utilizadas em aplicações de Processamento de Linguagem Natural (PLN), como análise de sentimentos, tradução automática e geração de texto. Além disso, são empregadas em tarefas de séries temporais, como previsão de séries financeiras e climáticas, e em sistemas de recomendação personalizada, onde são capazes de capturar padrões temporais complexos para realizar previsões mais precisas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3 Redes Adversárias Generativas

Redes Adversárias Generativas (do Inglês “Generative Adversarial Network”, GANs), são um tipo de modelo de Deep Learning composto por duas redes neurais: o **gerador** e o **discriminador**. Essas redes são treinadas em conjunto, com o objetivo de gerar dados que se assemelhem a amostras reais do conjunto de dados original. O gerador cria amostras sintéticas, enquanto o discriminador tenta distinguir entre amostras reais e sintéticas. Observe a figura:

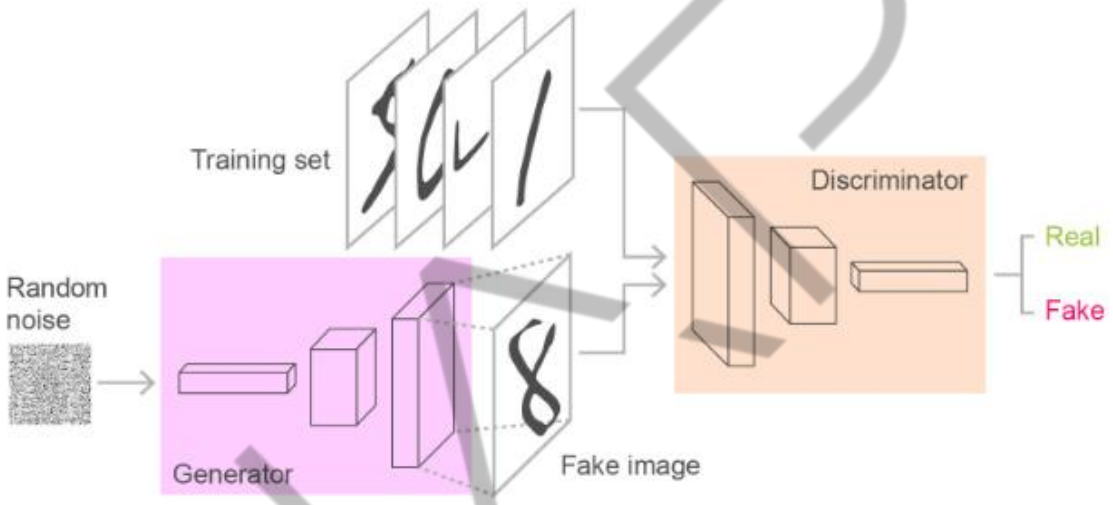

Figura 12 - Representação de uma GAN
Fonte: Deep Learning Book (2022), adaptado por FIAP (2025)

O processo de treinamento das GANs é iterativo e competitivo. O gerador tenta melhorar constantemente suas habilidades para gerar amostras cada vez mais realistas, enquanto o discriminador busca aprimorar sua capacidade de distinguir entre amostras reais e falsas. Essa competição leva a um equilíbrio dinâmico, onde o gerador aprende a gerar amostras que são cada vez mais indistinguíveis das reais.

As GANs têm uma ampla gama de aplicações, incluindo **geração de imagens**, **síntese de texto**, **geração de áudio** e muito mais. Elas são especialmente úteis em tarefas onde a disponibilidade de dados é limitada ou onde a coleta de dados reais é cara ou impraticável. No entanto, treinar GANs pode ser desafiador e requer técnicas avançadas de otimização e regularização para evitar problemas como colapso do modo e instabilidade durante o treinamento.

Este tipo de arquitetura de Deep Learning desempenha um papel crucial no avanço da **Inteligência Artificial Generativa** (Generative AI, ou GenAI), permitindo a

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

criação de conteúdos diversificados e personalizados. Além disso, elas impulsionam o desenvolvimento de novas técnicas de geração de dados e representam um passo significativo em direção à criação de sistemas de Inteligência Artificial capazes de entender e produzir informações complexas e realistas. Por outro lado, a geração de dados levanta questões e amplia o debate sobre seus usos maliciosos como *fake news*, ética em IA, e efeitos de vieses de dados.

3.4 Outras arquiteturas

Atualmente, as principais aplicações de Deep Learning se baseiam nas arquiteturas apresentadas. Existem outras arquiteturas profundas além delas, mas sua introdução está fora do escopo. Entre as demais arquiteturas, destacam-se:

- **Autoencoders:** são redes neurais usadas para aprendizado não supervisionado de representações de dados. Eles consistem em uma rede codificadora que reduz a dimensionalidade dos dados de entrada, e uma rede decodificadora que tenta reconstruir os dados de entrada originais a partir da representação codificada.
- **Redes Generativas Adversariais Variacionais (VAE-GAN):** são uma combinação entre GANs e autoencoders variacionais (VAEs). Elas são capazes de gerar imagens de alta qualidade e oferecem maior controle sobre a geração de amostras.
- **Redes Siamesas:** são arquiteturas que consistem em dois ramos idênticos que compartilham os mesmos pesos. Essas redes são utilizadas em tarefas de aprendizado por pares, como reconhecimento facial e tradução.
- **Redes Recorrentes Convolutivas (RCN):** são uma combinação de redes neurais convolucionais (CNNs) e redes neurais recorrentes (RNNs). Elas são eficazes em modelar sequências temporais em dados 2D, como séries temporais em imagens.

A título de curiosidade, a lista de outras arquiteturas de redes neurais (profundas ou não) e suas representações está na figura “Arquiteturas de Redes

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Neurais (Profundas ou não)”. A composição de uma lista completa é praticamente impossível, já que novas arquiteturas são inventadas ou aperfeiçoadas o tempo todo.

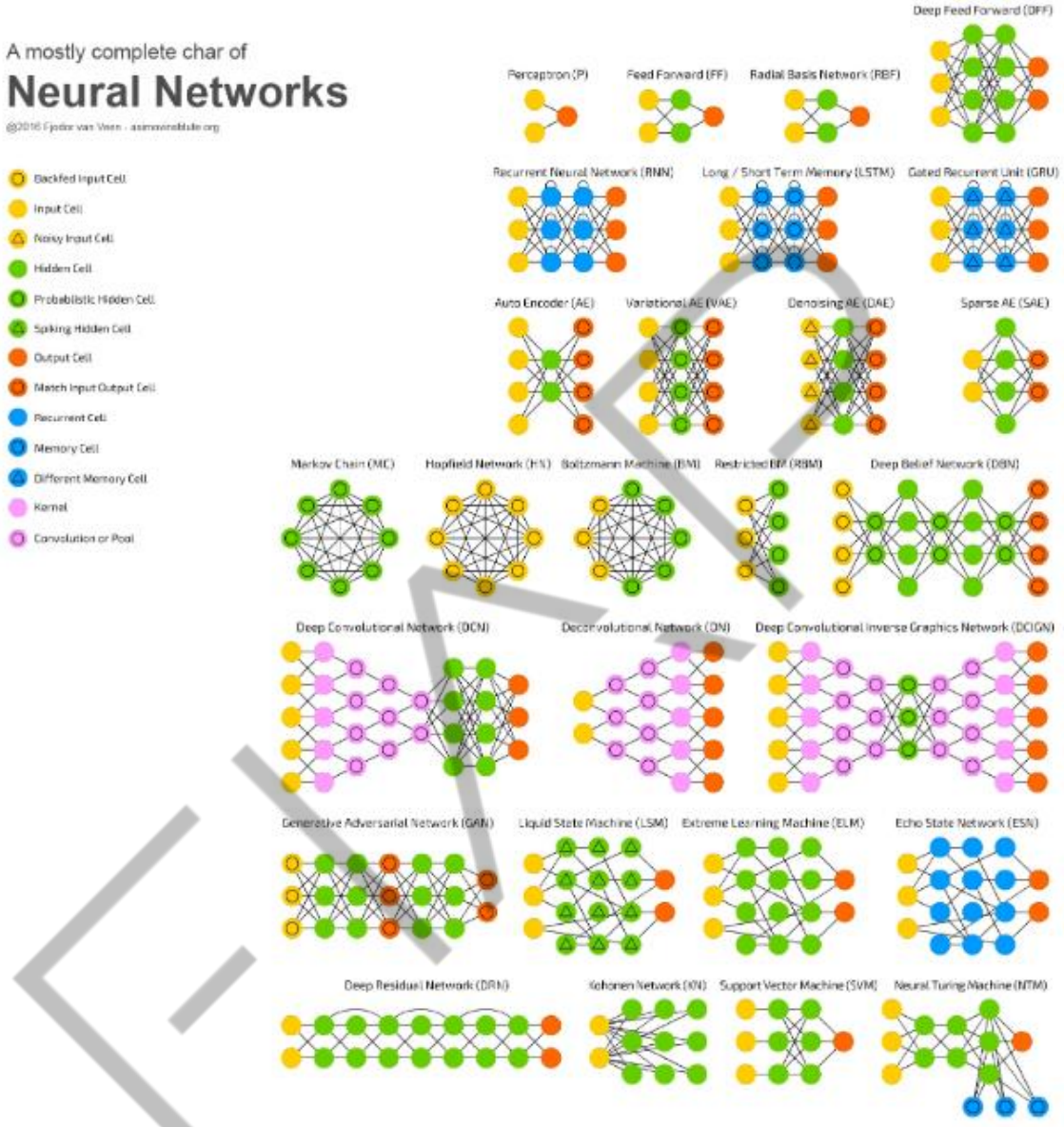

Figura 13 - Arquiteturas de Redes Neurais (Profundas ou não)
Fonte: The Asimov Institute (2017), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 ARQUITETURAS CONVOLUCIONAIS

4.1 Representação de uma imagem

Antes de prosseguirmos, é importante estarmos familiarizados com a representação computacional de uma imagem. Afinal, para um computador, o que é uma imagem?

Quando uma imagem é representada em um computador, ela é dividida em **pixels**, que são os pequenos pontos coloridos que formam a imagem.

Existem diversos espaços de características para representar os pixels. O mais comum é o **RGB**, no qual a imagem é representada por uma matriz tridimensional, onde as duas primeiras dimensões correspondem à **altura** e **largura** da imagem, e a terceira dimensão representa os três **canais de cores**: vermelho (R – *red*), verde (G – *green*) e azul (B – *blue*).

Cada canal representa a **intensidade de cor em uma escala de 0 a 255**:

- O valor (255, 0, 0) representa o vermelho puro;
- O valor (0, 255, 0) representa o verde puro;
- O valor (0, 0, 255) representa o azul puro.

A cor do pixel que enxergamos é a soma das representações dos três canais. Por exemplo, uma imagem de dimensões 100x100 pixels em RGB seria representada por uma matriz de tamanho 100x100x3. Cada elemento dessa matriz contém três valores, um para cada canal de cor, representando a intensidade de vermelho, verde e azul do pixel correspondente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

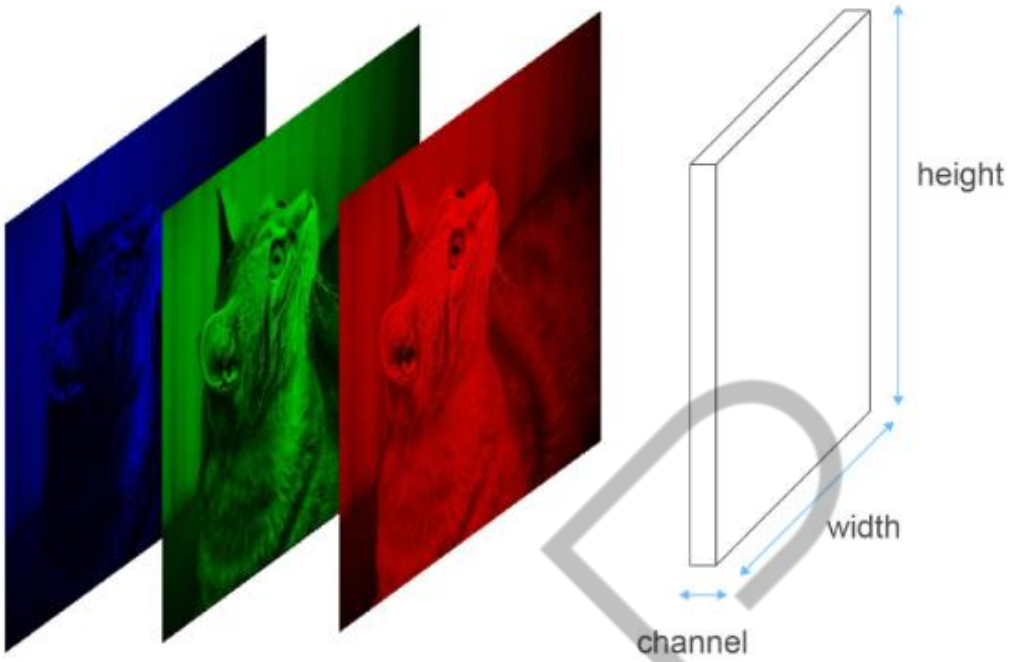

Figura 14 - Representação de uma imagem colorida em RGB
Fonte: Medium (2021), adaptado por FIAP (2025)

Para imagens em escala de cinza, há apenas um canal de cor, onde cada pixel é representado por um único valor de intensidade de cinza, variando de 0 (preto) a 255 (branco). Nesse caso, quanto maior o valor do pixel, mais claro ele é, e quanto menor, mais escuro.

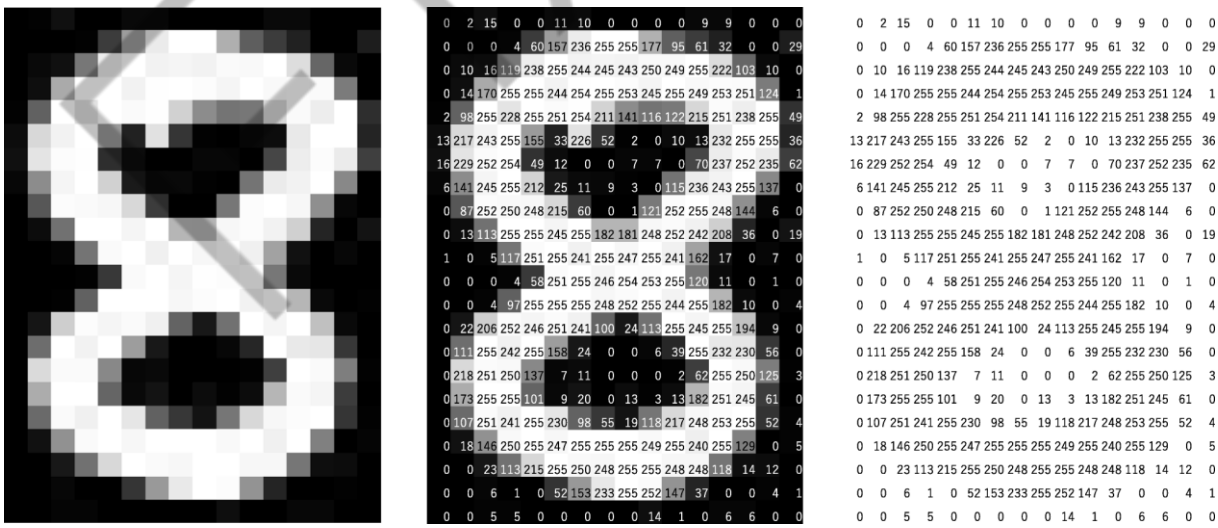

Figura 15 - Representação de uma imagem tons de cinza
Fonte: Medium (2020)

A figura “Representação de uma imagem tons de cinza” mostra uma imagem em escala de cinza representando o número “8” escrito à mão. Note que cada linha e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

coluna representa um pixel, expresso de 0 a 255. Os valores próximos à 0 representam o mais escuro possível e os valores próximos a 255 o mais claro possível.

Essa representação de 0 a 255 é uma forma de normalização dos valores dos pixels, onde 0 representa a ausência de cor e 255 representa a cor mais intensa. Isso facilita o processamento da imagem pelo computador e a manipulação dos valores de cor para realizar tarefas como detecção de bordas, reconhecimento de objetos e classificação de imagens.

4.2 Estrutura básica

A revolução das CNNs teve seu marco inicial com a AlexNet, vencedora da competição ImageNet de 2012. A AlexNet demonstrou uma capacidade surpreendente de reconhecimento de imagens em grande escala, impulsionada pela arquitetura profunda e pela aplicação eficiente de camadas convolucionais.

Uma CNN é composta por várias camadas, representada pela figura “Representação da AlexNet”, sendo as principais as camadas **convolucionais** (retângulos azuis), de **pooling** (os cones) e **totalmente conectadas** (FC – Fully Connected).

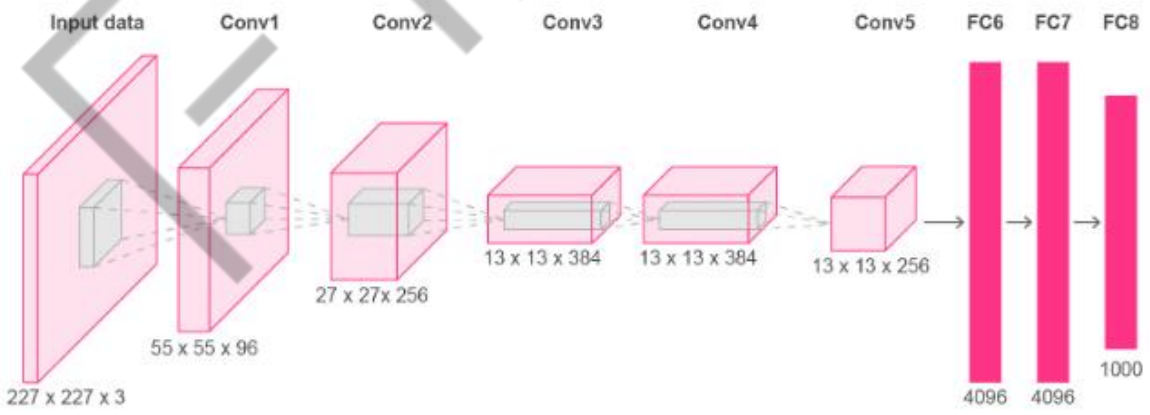

Figura 16 - Representação da AlexNet
Fonte: MDPI (2017), adaptado por FIAP (2025)

As **camadas convolucionais** são responsáveis por aprender padrões e características relevantes nas imagens, utilizando operações de convolução para aplicar filtros sobre a entrada. Em seguida, as **camadas de pooling** reduzem a

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

dimensionalidade dos dados, preservando as características mais importantes. Por fim, as **camadas totalmente conectadas** processam essas características para realizar a classificação final.

Não há a necessidade de extrair features por meio de descritores, pois a extração ocorre automaticamente a partir dos pixels da imagem. O treinamento da rede visa garantir que a CNN está aprendendo a extrair features cada vez mais relevantes para a tarefa de destino (exemplo: classificar imagens como contendo gatos ou cachorros).

A flexibilidade e a eficácia das CNNs revolucionaram o campo da inteligência artificial e continuam a ser uma área de intenso desenvolvimento e pesquisa.

4.3 Entendendo operações de Convolução e Pooling

Podemos afirmar que as CNNs são redes profundas para extração de features. Mas como esse processo funciona? Esta extração está baseada nas operações de **convolução** e **pooling**.

Intuição: imagina que você tem um montão de LEGO e quer construir uma casa. Primeiro, você pega uma peça e vai encaixando outra em cima dela, uma por uma, até formar uma parede. Essa é a convolução, é como ir montando tijolinho por tijolinho. Depois, quando a parede está grande, você percebe que existem alguns tijolinhos pequenos e iguais juntinhos. Então você substitui estes tijolinhos pequenos por um bloco maior. Isso é o pooling: juntar os tijolinhos parecidos para a casa ficar mais simples e rápida de montar.

Explicação matemática: convolução é uma operação matemática linear que, a partir de duas funções dadas, resulta em uma terceira que mede **a soma do produto dessas funções** ao longo da região subentendida pela superposição delas **em função do deslocamento** existente entre elas.

Operações em uma Rede Neural: a operação de **convolução** em uma CNN é uma técnica que consiste em deslizar um filtro (também chamado de kernel) sobre a imagem de entrada, multiplicando os valores dos pixels da imagem pelos valores do filtro e somando os resultados para obter um único valor na saída, formando uma nova imagem. Observe a figura:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

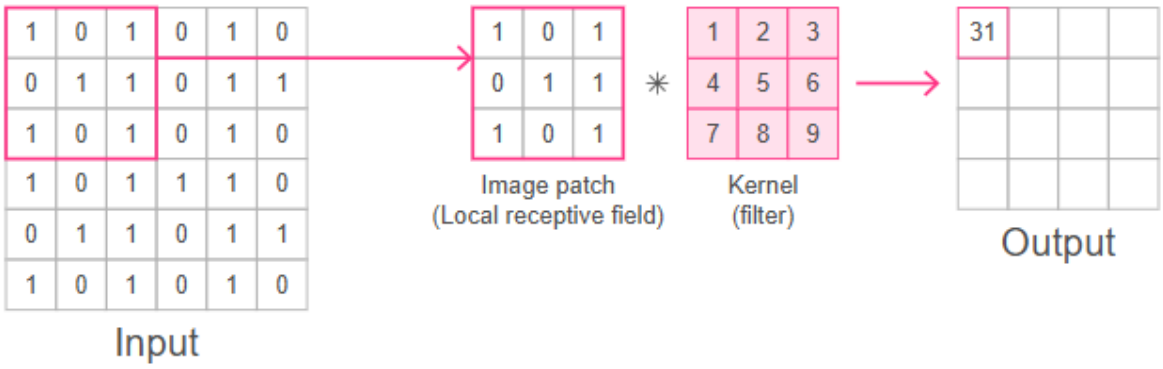

Figura 17 - Operação de convolução (1)
Fonte: SuperAnnotate (2023), adaptado por FIAP (2025)

Essa operação é repetida para diferentes regiões da imagem de entrada, produzindo uma imagem de saída que destaca padrões e características relevantes da imagem original.

Figura 18 - Operação de convolução (2)
Fonte: Towards Data Science (2020), adaptado por FIAP (2025)

Já o **pooling** é uma operação de subamostragem que reduz a dimensionalidade da imagem, preservando as características mais importantes. A operação de pooling, representada na figura “Operação de pooling”, divide a imagem em regiões e substitui cada região por um único valor, comumente o máximo ou a média dos valores da região. Isso ajuda a reduzir o custo computacional e o overfitting, além de fornecer invariância a pequenas variações na posição dos padrões na imagem.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

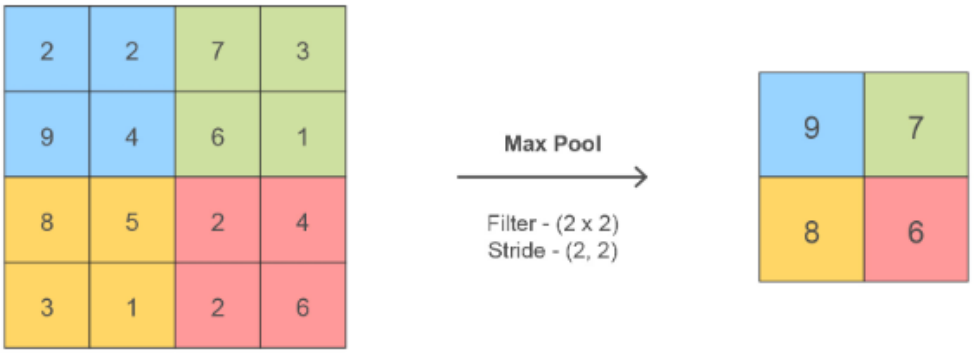

Figura 19 - Operação de pooling
Fonte: Geeksforgeeks (2025), adaptado por FIAP (2025)

A definição matemática de convolução indica um “deslocamento” entre as funções. Nas CNNs, este deslocamento é dado pela janela deslizante do kernel sobre a imagem. Considere a figura “Exemplo de convolução”, ela ilustra a operação de convolução entre uma imagem 4 (linhas) x 5 (colunas) e um kernel 2x2. A cada passo, parte da imagem é considerada uma função e o filtro do kernel outra função. Multiplicadas, elas produzem um escalar que está preenchendo outra matriz, a qual, após preenchida, será multiplicada pelo kernel novamente.

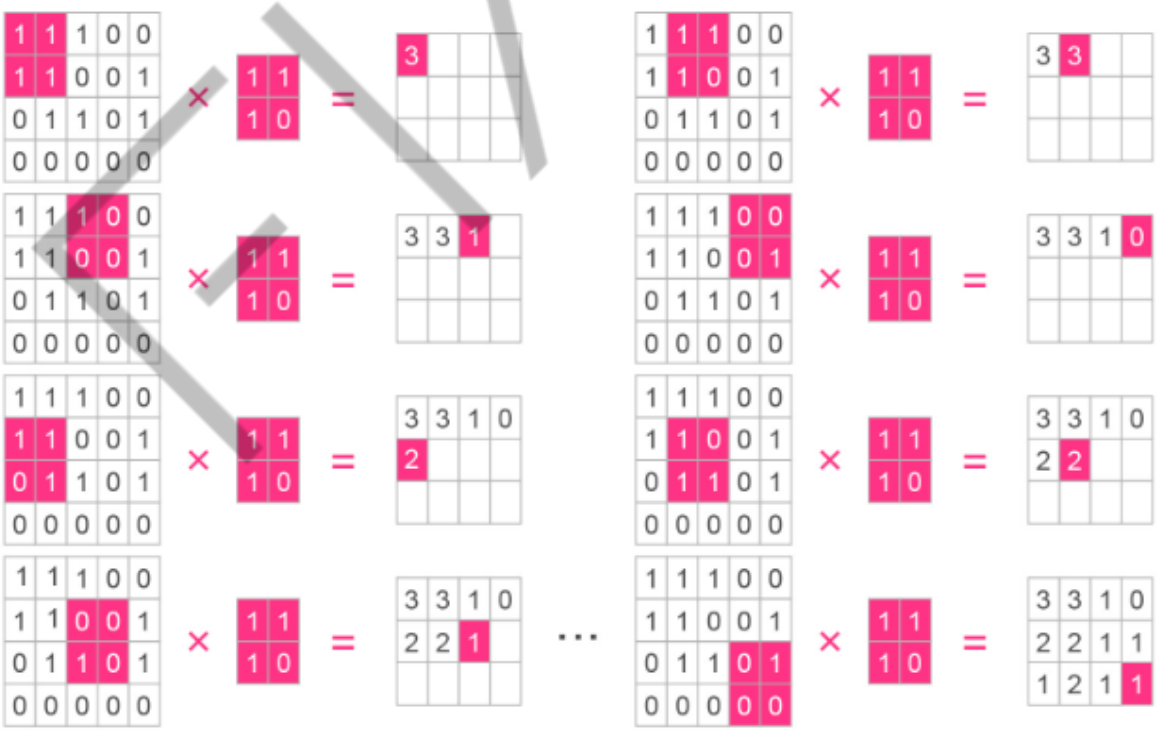

Figura 20 - Exemplo de convolução
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Note que a convolução está ocorrendo da esquerda para a direita com um passo de 1 posição tanto para coluna quanto para linha. No final da imagem temos a matriz resultante completamente preenchida.

Agora chegou a hora de fazer o pooling. A figura a seguir ilustra o processo iterativo. Uma vez estabelecido o tamanho da imagem resultante e das regiões a serem aplicadas o pooling, a operação mantém o valor mais representativo daquela região.

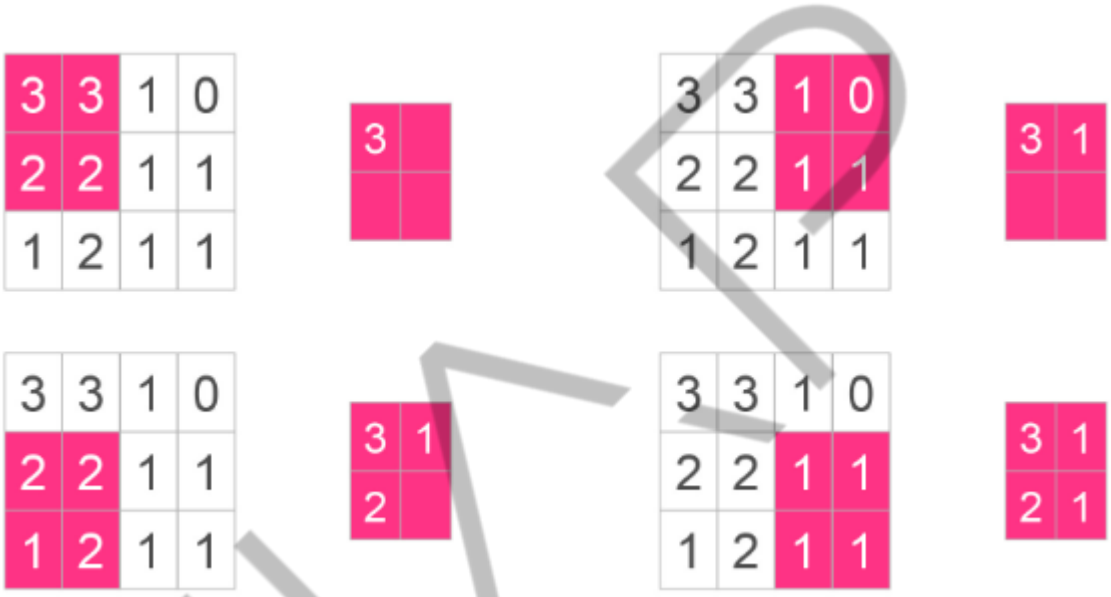

Figura 21 - Exemplo de pooling
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

Existem formas distintas de aplicar o pooling, todas visam reduzir a dimensionalidade das features extraídas da imagem. Os mais comuns são:

- **Max Pooling:** no max pooling, uma região (geralmente 2x2) é percorrida pela imagem de entrada e o **valor máximo** dentro dessa região é mantido na saída. Essa técnica enfatiza características dominantes e invariantes de posição, como bordas e texturas distintas, mantendo-as na saída.
- **Average Pooling:** no average pooling, uma região (novamente, geralmente 2x2) é percorrida pela imagem de entrada e a **média dos valores** dentro dessa região é calculada e mantida na saída. Esta técnica é mais suave e menos propensa a perder informações importantes, uma vez que considera todos os valores na região de pooling, em vez de apenas o máximo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Mas quem disse que o melhor kernel 2x2 para esta imagem deveria ser aquele ilustrado em laranja na figura “Componentes típicos de uma CNN”? Pois é, não há garantias que este kernel seja o mais indicado para todos os problemas de classificação de imagem. Portanto, ele é um parâmetro que precisa ser aprendido! De fato, ele é o peso da rede que precisa ser refinado durante o treinamento!

4.4 Configurações avançadas

Além do tamanho da convolução e a operação de pooling a ser aplicada, existem outros parâmetros a serem pensados na composição da arquitetura da CNN. Entre eles, os mais relevantes são o **padding** (preenchimento) e o **striding** (passo), que afetam o tamanho e a resolução das saídas das camadas convolucionais. Por fim, a escolha das **funções de ativação** influencia como a máquina aprende.

4.4.1 Padding e Striding

O **padding** é uma técnica usada para controlar o tamanho da saída após a aplicação da operação de convolução. Ele consiste em adicionar zeros ao redor da borda da entrada antes de aplicar a convolução. O padding pode ser “*valid*” (sem padding) ou “*same*” (padding suficiente para manter a altura e largura na saída). A figura a seguir mostra o padding de 1, incluindo uma borda de 1 pixel ao redor da imagem.

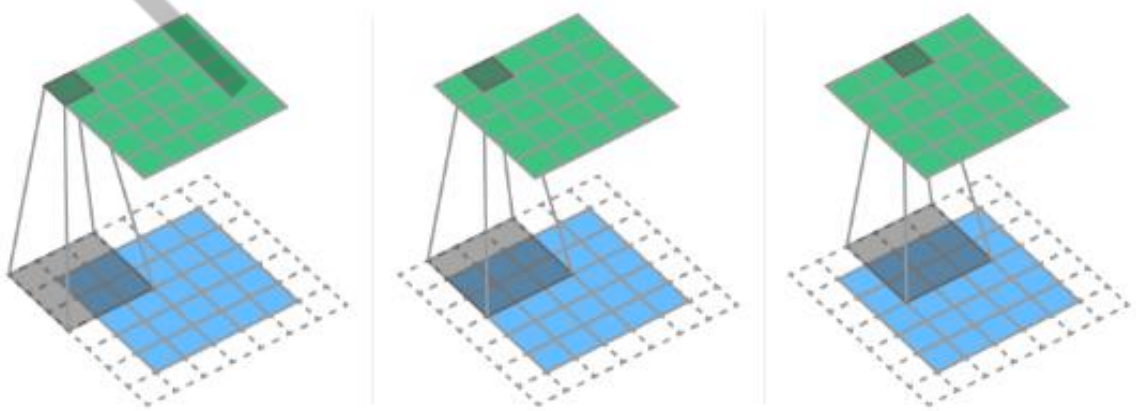

Figura 22 - Exemplo de padding
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O padding é importante para preservar informações nas bordas das imagens e evitar a diminuição do tamanho da saída após várias camadas de convolução.

O **striding** determina o **quanto o filtro se move pela entrada a cada passo durante a operação de convolução**. Um striding de 1 significa que o filtro se move pixel a pixel, enquanto um striding maior reduz a resolução da saída, pois o filtro salta mais pixels a cada passo. Um striding maior pode ajudar a reduzir o tempo de computação e o custo computacional, mas pode levar à perda de informações finas nas camadas convolucionais. A figura a seguir mostra um striding de tamanho 2. Note que o valor é aplicado tanto na largura quanto na altura da imagem.

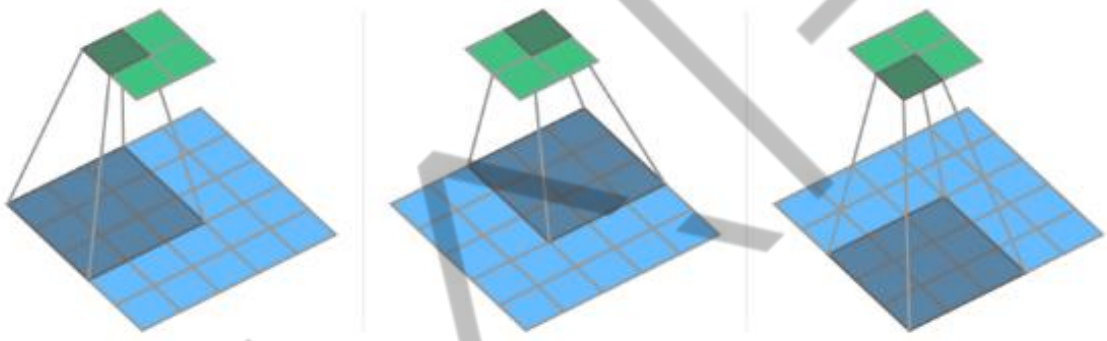

Figura 23 - Exemplo de striding
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

A configuração adequada do padding e do striding pode ter impactos significativos no desempenho e na eficácia de uma CNN. O uso adequado de padding pode ajudar a preservar informações nas bordas das imagens e manter o tamanho da saída desejado. Por outro lado, um striding maior pode reduzir a resolução espacial da saída, o que pode ser útil para reduzir o custo computacional e a dimensionalidade dos dados, mas pode levar à perda de informações importantes.

4.4.2 Funções de ativação

As funções de ativação desempenham um papel crucial nas redes neurais por introduzirem não linearidades, permitindo que a rede aprenda e represente relações complexas nos dados. A tabela a seguir **lista as funções de ativação mais comuns e o impacto de seu uso em CNNs**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Função de Ativação	Prós	Contras	Melhores práticas
ReLU (Rectified Linear Unit)	Simplicidade computacional, ajuda a mitigar o problema de *vanishing gradients*, acelera a convergência em comparação com funções sigmoidais.	Pode levar à “dying ReLU problem”, onde os neurônios param de aprender devido a valores de entrada negativos constantemente resultando em saídas zero.	É amplamente utilizado em camadas convolucionais devido à sua eficácia e eficiência computacional.
Leaky ReLU	Resolvem o problema do “dying ReLU”, permitindo um pequeno gradiente para valores negativos.	Pode resultar em valores inconsistentes para gradientes negativos.	É útil em casos em que a ReLU padrão não converge bem.
ELU (Exponential Linear Unit)	Combate o problema do “dying ReLU”, produzindo uma saída suave para valores negativos e permitindo gradientes suaves.	Mais computacionalmente caro devido ao uso de funções exponenciais.	Pode ser eficaz em redes mais profundas e em problemas que exigem suavidade nas ativações.
Sigmoid e Tangente Hiperbólica (tanh)	Produzem saídas no intervalo limitado (0 a 1 para sigmoid e -1 a 1 para tanh), útil para problemas de classificação binária.	Propensos ao problema de *vanishing gradients*, que podem resultar em valores saturados nos gradientes.	Principalmente utilizados em camadas de saída para problemas de classificação binária.
Linear	Simplicidade, útil em camadas de saída para problemas de regressão.	Não introduz não linearidades, limitando a capacidade de aprendizado da rede.	Usado em camadas de saída para problemas de regressão, onde as previsões precisam ser contínuas.

Quadro 1 - Funções de ativação para CNNs
Fonte: Elaborado pelo autor (2024)

Em geral, a escolha da função de ativação depende do problema específico, da arquitetura da rede e da experimentação para determinar qual função funciona melhor para o caso em questão. Experimentação e ajuste fino são fundamentais para determinar a melhor função de ativação em uma CNN.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.5 Terminologia e evolução

A terminologia das CNNs pode variar de acordo com diferentes características e dimensões. Os termos mais importantes são **profundidade** (*depth*), **largura** (*width*) e **resolução** (*resolution*). Estes conceitos estão ilustrados na figura “Terminologias de CNNs”, sendo comparados com uma rede baseline.

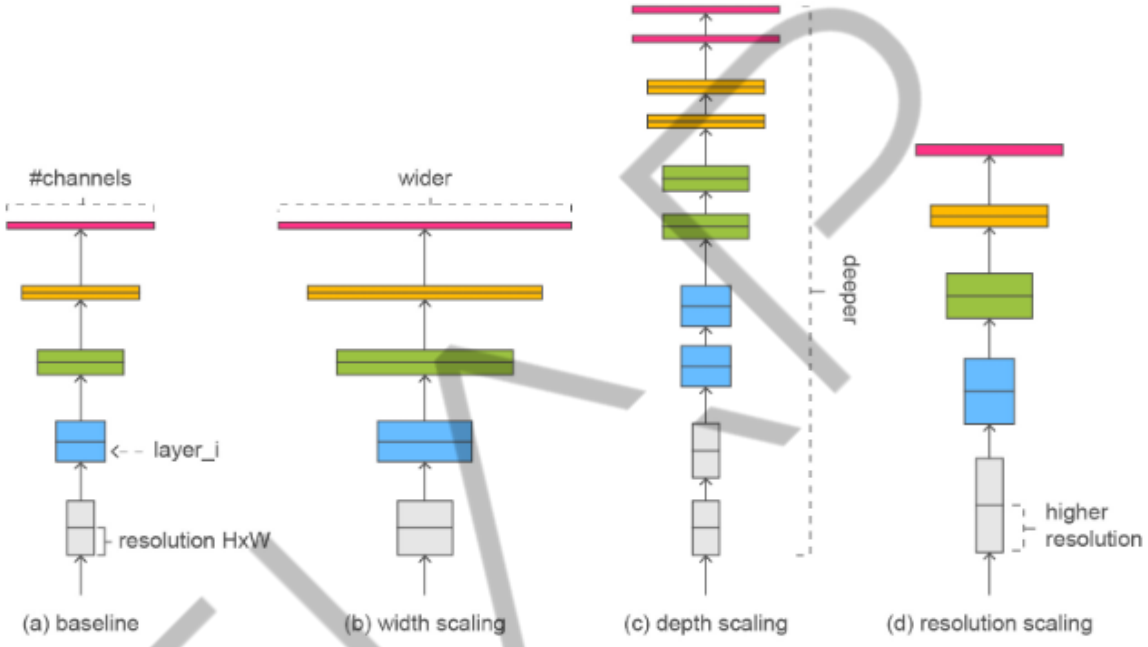

Figura 24 - Terminologias de CNNs
Fonte: MDPI (2022), adaptado por FIAP (2025)

Tais conceitos se referem a:

- **Redes Profundas:** redes com **várias camadas**, que podem incluir dezenas a centenas de camadas. Essas redes têm a capacidade de aprender representações hierárquicas de recursos nos dados, tornando-as ideais para tarefas complexas que exigem uma compreensão profunda das características intrínsecas dos dados.
- **Redes Largas**: redes com **grande número de neurônios em cada camada**. Essas redes têm uma grande capacidade de aprendizado, pois podem capturar uma ampla gama de características dos dados. No entanto, elas também podem ser mais propensas a overfitting, especialmente quando os conjuntos de dados são pequenos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Redes de Maior Resolução:** redes que operam em **imagens de alta resolução** ou em outras formas de dados com alta dimensionalidade. Essas redes podem exigir camadas especiais de pré-processamento ou arquiteturas adaptadas para lidar com a complexidade dos dados de alta resolução.

A evolução das arquiteturas de CNNs tem sido marcada por avanços significativos ao longo dos anos, tanto em termos de profundidade quanto de resolução e amplitude.

Inicialmente, as arquiteturas de CNNs eram relativamente rasas e simples, como a AlexNet que introduziu o conceito de convoluções em profundidade e pooling. Com o tempo, arquiteturas mais avançadas foram desenvolvidas para lidar com desafios mais complexos e melhorar o desempenho em tarefas de visão computacional. Se traçarmos a precisão de todos os trabalhos relatados no ImageNet, obteríamos o gráfico da seguinte figura:

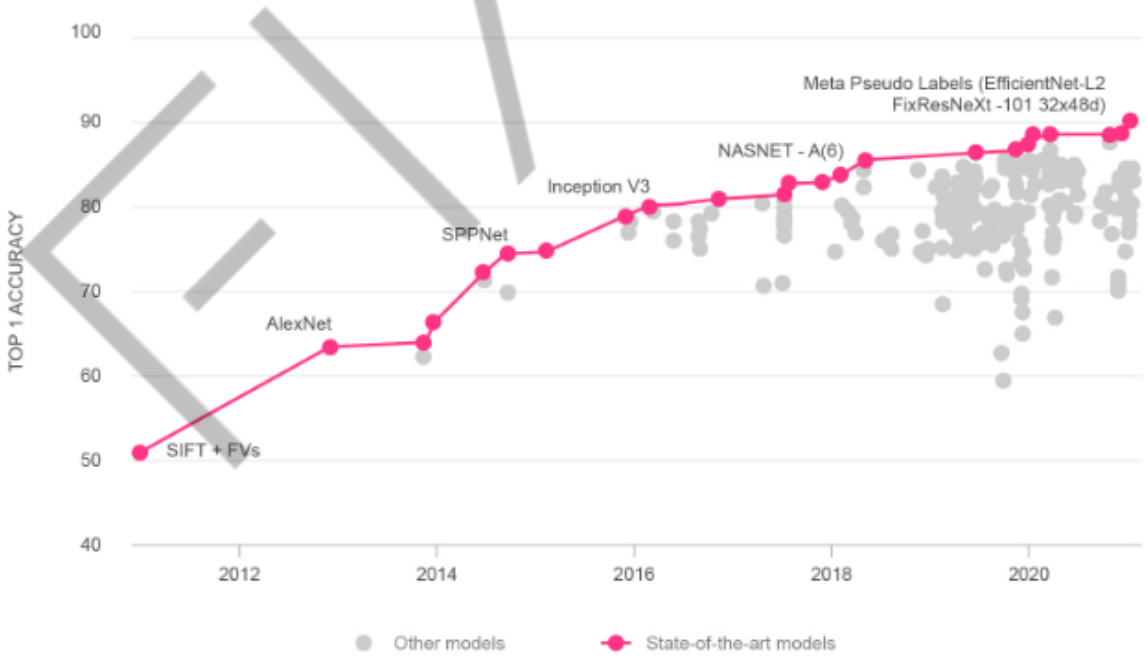

Figura 25 - Benchmark de CNNs
Fonte: The AI Summer (2021), adaptado por FIAP (2025)

O tamanho das redes cresceu e variou enormemente ao longo dos anos, com arquiteturas cada vez mais profundas e complexas. Por outro lado, redes mais simples foram propostas e conseguiram resultados extraordinários, ao passo de reduzirem o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

esforço computacional para treinamento. Para fornecer outra visão geral, seria possível capturar as CNNs de melhor desempenho até 2018 em uma única imagem, presente na figura a seguir.

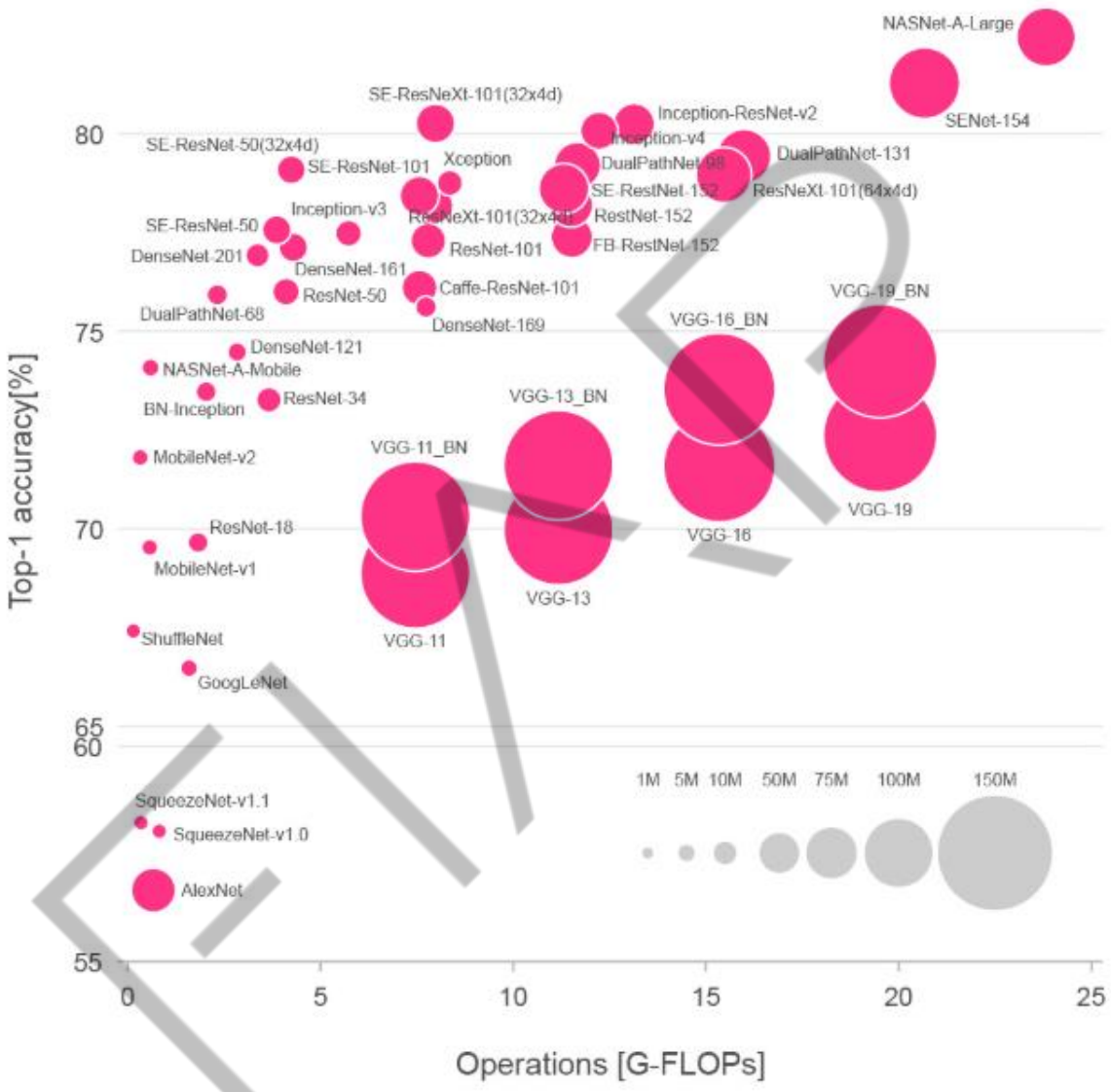

Figura 26 - Overview das arquiteturas CNNs até 2018
Fonte: The AI Summer (2021), adaptado por FIAP (2025)

Podemos notar pela figura anterior que a AlexNet — totalmente revolucionária em 2012 — está completamente obsoleta em 2018. Repare também que à medida que as redes foram ficando mais complexas, a performance aumentou. No entanto, mesmo as redes menores conseguem alcançar desempenho de estado da arte na competição. Portanto, profundidade não é sinônimo de desempenho, e a forma como

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

as redes são conectadas e configuradas influenciam enormemente os resultados atingidos.

Algumas das arquiteturas mais modernas e influentes incluem:

- **VGGNet:** introduziu uma arquitetura profunda e uniforme com múltiplas camadas convolucionais, ajudando a estabelecer o padrão para redes mais profundas.
- **GoogLeNet (Inception)**: apresentou módulos de convolução inception, que permitem uma exploração eficiente de várias escalas de recursos e reduzem a dimensionalidade dos mapas de características intermediárias.
- **ResNet**: introduziu blocos residuais, que permitem o treinamento de redes ainda mais profundas, mitigando o problema de *vanishing gradients* por meio de conexões de atalho.
- **EfficientNet**: utiliza escalonamento composto para otimizar o equilíbrio entre profundidade, largura e resolução da rede, alcançando melhor desempenho em comparação com arquiteturas convencionais.

Essas arquiteturas mais modernas incorporam técnicas inovadoras para melhorar a eficiência e o desempenho das CNNs, permitindo avanços significativos em uma ampla gama de aplicações de visão computacional, como segmentação e identificação de objetos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 TREINAMENTO DE REDES NEURAIS CONVOLUCIONAIS

Nesta etapa, exploraremos aspectos cruciais do treinamento das CNNs, essenciais para extrair o máximo desempenho desses modelos em uma variedade de tarefas de visão computacional.

A **preparação adequada dos dados** é fundamental para garantir que as CNNs sejam capazes de aprender padrões significativos nos dados de entrada, enquanto **técnicas de regularização** são essenciais para evitar o overfitting e melhorar a generalização do modelo. Além disso, a **otimização de hiperparâmetros** desempenha um papel crucial na busca pela configuração ideal do modelo, maximizando sua capacidade de aprender e generalizar a partir dos dados disponíveis. Por fim, abordaremos também o treinamento de **redes 3D (convolução sobre volumes)**, que permite estender as capacidades das CNNs para dados volumétricos, abrindo novas possibilidades de aplicação em áreas como imagens médicas e processamento de vídeo.

Ao compreender e dominar esses aspectos do treinamento de CNNs, podemos aproveitar ao máximo o potencial do Deep Learning em diversas tarefas.

5.1 Preparação de dados

A preparação de dados para CNNs envolve várias etapas, começando pela coleta e aquisição dos dados brutos. Em seguida, os dados geralmente passam por uma etapa de pré-processamento, onde é realizado operações como **redimensionamento** para um tamanho padrão, **normalização** para garantir que todas as características tenham a mesma escala e, se necessário, **aumento de dados** para aumentar a variabilidade dos dados de treinamento.

Além disso, a **segmentação** pode ser aplicada para isolar regiões de interesse em imagens complexas. Outras técnicas de pré-processamento incluem a **remoção de ruído** e **artefatos indesejados**, bem como a correção de distorções ou irregularidades nos dados de entrada.

Uma consideração importante na preparação de dados para CNNs é a **organização adequada dos dados em conjuntos de treinamento, validação e**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

teste. É essencial que esses conjuntos sejam representativos da distribuição dos dados do mundo real e que sejam suficientemente grandes para permitir uma avaliação precisa do desempenho do modelo. Pode ser útil considerar a presença de classes desbalanceadas e garantir que o conjunto de dados inclua exemplos suficientes de cada classe.

5.2 Técnicas de regularização

No contexto das CNNs, a regularização é essencial para evitar que o modelo se torne muito complexo e superajustado aos dados de treinamento, garantindo, assim, que ele generalize bem para novos dados e produza resultados mais confiáveis e robustos.

Uma técnica comum de regularização é a **Dropout**, que consiste em desligar aleatoriamente um certo número de unidades de neurônios durante o treinamento, reduzindo assim a co-adaptação entre os neurônios e melhorando a generalização do modelo. Outras técnicas de regularização incluem: a **Regularização L1**, que adiciona uma penalidade baseada na norma L1 dos pesos; e a **Regularização L2**, que adiciona um termo à função de custo do modelo, penalizando os pesos grandes e incentivando pesos menores, o que reduz a complexidade do modelo.

Além disso, a **Normalização em Lotes (*Batch Normalization*)** é frequentemente utilizada como uma forma de regularização, normalizando as ativações de cada camada do modelo. Isso ajuda a estabilizar o treinamento e reduzir a dependência da inicialização dos pesos.

Finalmente, a **Redução da Taxa de Aprendizado** diminui gradualmente a taxa de aprendizado ao longo do treinamento para permitir uma busca mais precisa no espaço de parâmetros.

5.3 Otimização de hiperparâmetros

A otimização de hiperparâmetros é uma etapa crucial no desenvolvimento de modelos de CNNs, pois determina a configuração ideal do modelo, como a taxa de aprendizado, o tamanho do kernel da convolução, o número de camadas, padding,

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

striding e tantos outros. A escolha adequada desses hiperparâmetros pode resultar em um modelo mais eficiente e com melhor desempenho.

Uma abordagem comum para otimizar os hiperparâmetros é realizar uma busca em **grade** (*grid search*) ou **busca aleatória** (*random search*) sobre um espaço pré-definido de hiperparâmetros. Nesse processo, diferentes combinações de hiperparâmetros são avaliadas por meio de validação cruzada em um conjunto de dados de validação, e a combinação que leva ao melhor desempenho é selecionada.

Outra técnica é a **otimização bayesiana**, que utiliza métodos probabilísticos para encontrar a melhor combinação de hiperparâmetros de forma mais eficiente do que a busca em grade ou aleatória. Essa abordagem tende a convergir mais rapidamente para a região do espaço de hiperparâmetros que ocasiona melhores resultados.

Além disso, técnicas como o **uso de regularização** e normalização em lotes também podem ser consideradas como parte do processo de otimização de hiperparâmetros, pois afetam diretamente o desempenho do modelo.

Assim como em todo modelo de Machine Learning, este é um processo iterativo muitas vezes computacionalmente intensivo, mas essencial para desenvolver modelos de alto desempenho e com boa capacidade de generalização. É importante explorar diferentes abordagens e técnicas para encontrar a configuração ideal.

5.4 Operações sobre volumes

Em CNNs de redes RGB, os volumes de dados representam imagens tridimensionais, onde cada canal (R, G, B) corresponde a uma dimensão. As operações sobre esses volumes incluem convoluções tridimensionais, onde um filtro é aplicado em todas as três dimensões do volume de entrada simultaneamente. Durante a convolução, o filtro é deslizado ao longo de todas as três dimensões, capturando padrões espaciais em todas as camadas de cores da imagem.

Além disso, as operações de pooling podem ser realizadas em cada canal individualmente ou em todo o volume. O pooling reduz a dimensionalidade do volume, preservando as características mais relevantes. Assim como nas CNNs tradicionais, as CNNs de redes RGB também podem usar padding e striding para controlar o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

tamanho e a taxa de redução do volume ao longo das operações de convolução e pooling. Essas operações são essenciais para extrair e preservar características importantes das imagens RGB durante o processo de aprendizado da rede.

Para calcular uma célula de saída, execute a convolução em cada canal correspondente e, em seguida, some o resultado.

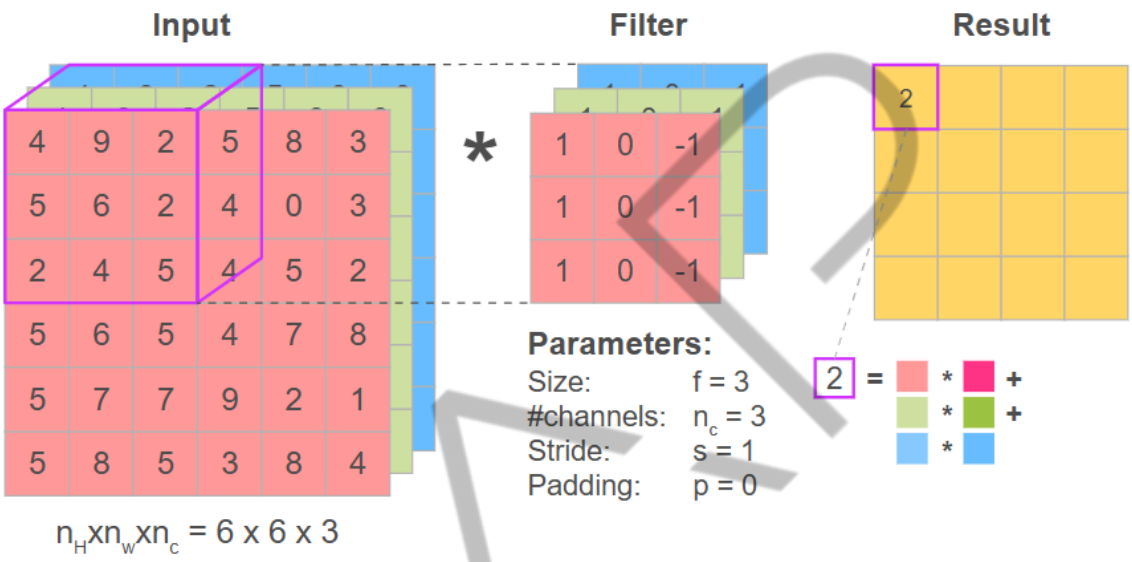

Figura 27 - Operação de Convolução em Volume
Fonte: IndoML (2018), adaptado por FIAP (2025)

No caso da representação da figura “Operação de Convolução em Volumes”, o número total de multiplicações a serem calculadas para atingir o resultado é (4 x 4) x (3 x 3 x 3) = 432.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 LIDANDO COM DESAFIOS EM REDES NEURAIS CONVOLUCIONAIS

Toda rede neural pode sofrer overfitting ou underfitting, e com as CNNs isso não é diferente. Ao longo do capítulo, vamos apresentar hiperparâmetros que auxiliam as redes convolucionais a extraírem cada vez mais informações dos dados (evitar underfitting), mas sempre ponderando sobre seu poder de generalização (evitar overfitting).

Ainda assim, as CNNs padecem de um grande problema: **a necessidade de grandes quantidades de dados para viabilizar seu treinamento**. Em alguns contextos, nem sempre é viável obter novos dados, seja pelos custos envolvidos ou pela “raridade” do fenômeno registrado na imagem.

Para contornar a baixa disponibilidade de dados e ainda assim obter bons resultados nos treinamentos das CNNs, existem duas estratégias comumente aplicadas, conhecidas como: **aumentação da base de dados** e **transferência de aprendizado**.

6.1 Técnicas de ampliação da base de dados

A aumentação da base de dados (*data augmentation*) é uma técnica crucial para melhorar o desempenho e a generalização de modelos de Deep Learning, incluindo CNNs. Essa técnica consiste em criar amostras de treinamento a partir das amostras originais, aplicando transformações aleatórias nos dados de entrada. Isso aumenta a diversidade dos dados de treinamento e ajuda a reduzir o overfitting.

Existem várias técnicas de ampliação de dados comumente usadas em problemas de visão computacional, **como rotação, translação, espelhamento horizontal e vertical, zoom, alteração de brilho e contraste, corte aleatório e adição de ruído**. Essas transformações são aplicadas de forma aleatória e controlada durante o treinamento do modelo, permitindo que ele aprenda a reconhecer objetos em diferentes posições, orientações e condições de iluminação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

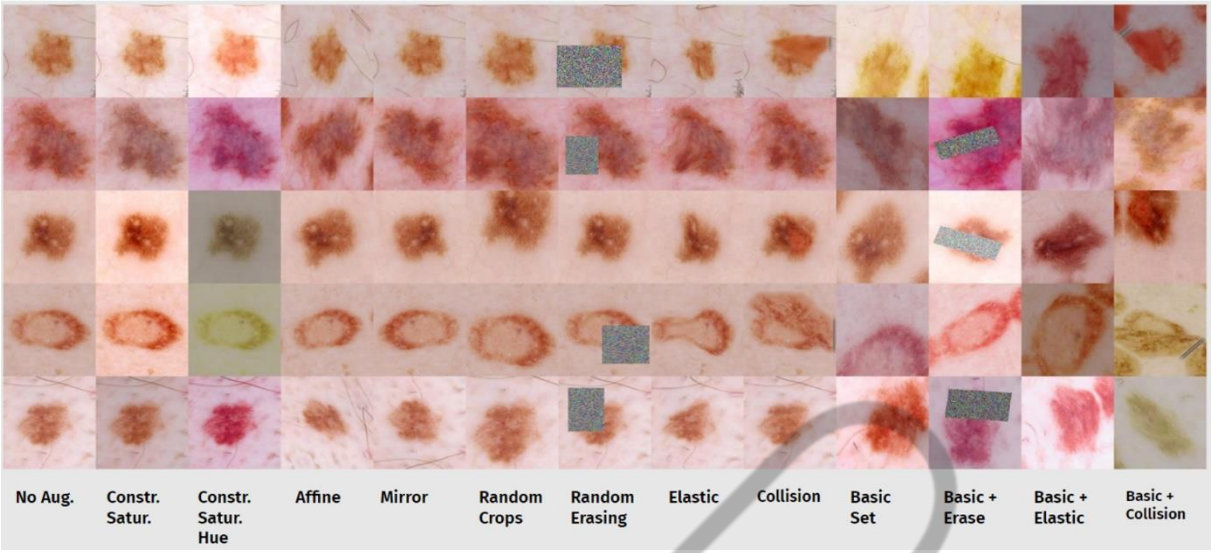

Figura 28 - Exemplos de aumentação de dados
Fonte: arxiv (2024)

Veja o resultado da aplicação de tais técnicas de aumentação na figura anterior. Cada linha se refere à mesma imagem de lesão de pele, sendo que em cada coluna está o resultado de uma técnica de aumentação, eventualmente combinadas. A primeira coluna representa a imagem original, sem aumentação alguma.

Dando um passo além, no sentido de usar técnicas mais sofisticadas, é possível **treinar GANs (Generative Adversarial Network — Redes Generativas Adversárias) para gerar amostras sintéticas que são indistinguíveis das amostras reais**.

Isso permite criar um número ilimitado de exemplos de treinamento adicionais, aumentando significativamente a diversidade e a quantidade de dados disponíveis para treinar modelos de Deep Learning, conforme ilustrado na figura a seguir. As quatro lesões desta figura são falsas, e foram geradas por meio de uma GAN.

Figura 29 - Exemplos imagens geradas com GANs
Fonte: arxiv (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Outra vantagem do uso das GANs é sua capacidade de aprender a distribuição subjacente dos dados de treinamento e gerar amostras que capturam as características essenciais dos dados reais. Isso ajuda a evitar o overfitting e melhora a capacidade de generalização do modelo.

No entanto, é importante observar que treinar GANs pode ser desafiador e requer cuidadosa sintonia de hiperparâmetros, além de grandes conjuntos de dados de treinamento. Além disso, é essencial garantir que as amostras geradas sejam de alta qualidade e representem adequadamente a distribuição dos dados reais.

6.2 Transfer Learning e Fine-tuning

Transfer Learning e Fine-tuning são técnicas essenciais em Deep Learning, especialmente em CNNs, que permitem aproveitar o conhecimento adquirido em uma tarefa para melhorar o desempenho em outra tarefa relacionada.

As camadas convolucionais e de pooling de uma CNN funcionam basicamente como um extrator de features, as quais podem ser aprendidas por qualquer método de Machine Learning, incluindo redes neurais totalmente conectadas.

O **Transfer Learning** envolve o uso de um modelo pré-treinado em uma grande base de dados, geralmente em uma tarefa semelhante àquela que estamos enfrentando. O modelo pré-treinado é então ajustado (ou “transferido”) para a nova tarefa, mantendo os pesos aprendidos nas camadas mais profundas e ajustando apenas as camadas finais para se adaptarem aos novos dados.

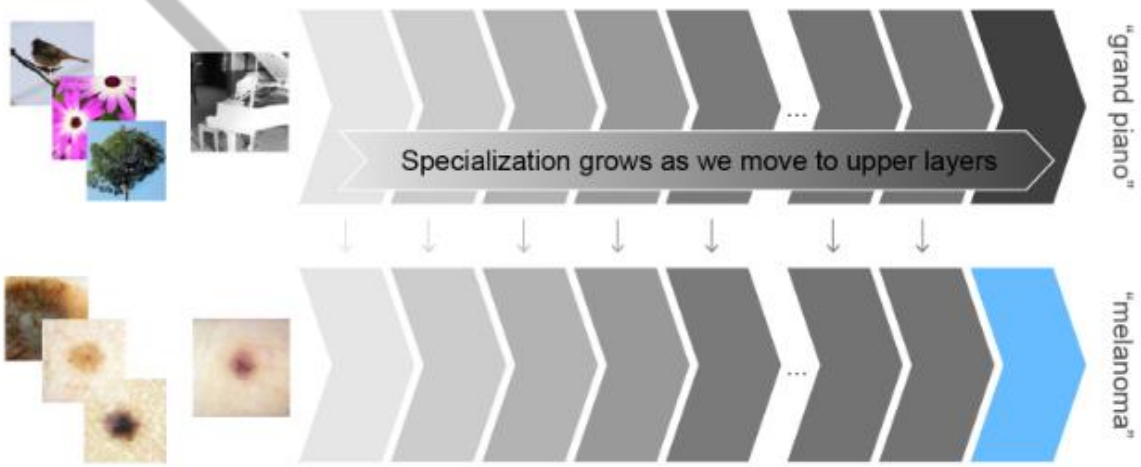

Figura 30 - Conceito de Transfer Learning
Fonte: arxiv (2024), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura “Conceito de Transfer Learning”, imagine que uma CNN foi treinada do zero em uma base de dados muito grande (por exemplo: ImageNet) para aprender a reconhecer objetos quaisquer. A mesma arquitetura foi usada para classificar imagens de lesões de pele entre “melanoma” ou “não melanoma”. Os pesos treinados da primeira rede foram usados para extrair características do segundo cenário, e apenas as últimas camadas (de classificação) foram retreinadas para ajuste à tarefa de destino.

Por outro lado, o **Fine-tuning** é uma técnica que envolve o ajuste fino de um modelo pré-treinado em uma tarefa específica. Isso geralmente envolve o descongelamento de algumas camadas do modelo pré-treinado e o re-treinamento do modelo em novos dados, com uma taxa de aprendizado menor para evitar grandes alterações nos pesos já aprendidos. Em outras palavras, é como se a rede fosse retreinada na tarefa de destino, porém não partindo de pesos aleatórios, mas de pesos que comprovadamente produzem boas features.

A importância dessas técnicas reside na capacidade de **reduzir o tempo e os recursos necessários para treinar modelos a partir do zero**, além de **melhorar o desempenho do modelo** em tarefas com conjuntos de dados menores. É importante mencionar que elas são altamente recomendadas quando temos um conjunto de dados pequeno ou limitado, pois permitem que o modelo se beneficie do conhecimento prévio adquirido em tarefas semelhantes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 HANDS ON: REDES NEURAIS PROFUNDAS

7.1 Deep Learning em Python

Agora que conhecemos o Pipeline de Deep Learning, está na hora de praticarmos sua implementação. Nesta atividade ainda não criaremos redes neurais profundas, mas introduziremos o uso do TensorFlow, contrastando-o com o Scikit-learn. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 2a), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Dica: para esta parte do código, utilizaremos um dataset já salvo na memória do TensorFlow, por isso não há arquivos externos associados!

Utilizaremos a base “Fashion MNIST” já persistida no TensorFlow. Criada por Zalando (uma empresa de e-commerce europeia), esta base é uma coleção de imagens de artigos de moda, composta por 70.000 imagens em tons de cinza, divididas em um conjunto de treinamento de 60.000 imagens e um conjunto de teste de 10.000 imagens. Cada imagem tem uma resolução de 28x28 pixels e está associada a um label de uma das 10 classes, representando diferentes tipos de roupas e acessórios. O objetivo é criar um modelo preditivo baseado em rede neural que possa classificar corretamente as imagens em suas respectivas categorias.

Vamos começar com os *imports*:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import tensorflow as tf
from tensorflow import keras

from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

import numpy as np
import matplotlib.pyplot as plt
```

Código-fonte 1 - Imports necessários
Fonte: Elaborado pelo autor (2024)

A célula anterior não produz saídas. Agora, vamos puxar o dataset da memória do TensorFlow. Como mencionado, já existe uma proposta de divisão das imagens em treino e teste:

```
# Carregar o dataset
fashion_mnist = keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels)
= fashion_mnist.load_data()

print()
print(train_images.shape)
print(len(train_labels))
print(train_labels)
print(test_images.shape)
print(len(test_labels))
```

Código-fonte 2 - Carregando o dataset Fashion MNIST
Fonte: Elaborado pelo autor (2024)

Ao executar o código anterior, você verá a saída da figura “Saída da execução do Código-fonte 2”. A primeira linha é o formato da variável que armazena o “treino”. Ela é um array tridimensional, composto por 60 mil imagens de 28 por 28 pixels. A segunda linha é o tamanho da lista de labels do treino (e há 60 mil labels, como o esperado). A terceira linha é uma exibição parcial dos labels do treino (notem as classes sendo expressas de 0 a 9). A quarta linha é o formato da variável que armazena o “teste”. Ela é um array tridimensional, composto por 10 mil imagens de 28 por 28 pixels. Concluindo, existem 10 mil labels para as imagens de teste, conforme o esperado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
(60000, 28, 28)
60000
[9 0 0 ... 3 0 5]
(10000, 28, 28)
10000
```

Figura 31 - Saída da execução do Código-fonte 2
Fonte: Elaborado pelo autor (2024)

Para facilitar a interpretação humana dos labels, faremos uma correspondência entre cada número de 0 a 9 ao nome textual da classe. O código “Explicitando o nome dos labels do dataset” faz esse “DE-PARA” via posições de uma lista, contendo os nomes das classes de acordo com a documentação da base de dados. O código não produz saída.

```
# Salva as classes
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

Código-fonte 3 - Explicitando o nome dos labels do dataset
Fonte: Elaborado pelo autor (2024)

Podemos exibir algumas imagens com o código a seguir, gerando a figura “Saída da execução do Código-fonte 4”.

```
# Exibir algumas imagens
plt.figure(figsize=(10,10))
for i in range(25):
  plt.subplot(5, 5, i+1)
  plt.xticks([])
  plt.yticks([])
  plt.imshow(train_images[i])
  plt.xlabel(class_names[train_labels[i]])
plt.show()
```

Código-fonte 4 - Exibindo algumas imagens do dataset
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 32 - Saída da execução do Código-fonte 4
Fonte: Elaborado pelo autor (2024)

Como em todo modelo de Machine Learning, é uma boa prática normalizar os dados trazendo-os para uma escala comum. Toda imagem é representada por uma matriz de pixels com valores de 0 até 255. Neste sentido, podemos entender que as imagens já possuem dados normalizados por natureza, pois todos os pixels já estão expressos sob a mesma escala.

No entanto, sabemos que as redes neurais convergem melhor se os dados forem escalados entre 0 e 1 ou entre -1 e +1. Já que todos os valores vão até, no máximo, 255, podemos facilmente normalizá-los dividindo-os por 255. Portanto:

```
# Normalizar os dados
train_images = train_images / 255.0
test_images = test_images / 255.0
```

Código-fonte 5 - Normalizando os dados
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Criaremos dois modelos preditivos baseados em redes neurais: o primeiro com Sklearn e o segundo com TensorFlow. Em ambos os casos as imagens bidimensionais precisam ser representadas como vetores unidimensionais para alimentar as redes. Este procedimento é chamado de “achatamento” (*flattening*). Basicamente ocorre a concatenação das linhas (ou colunas) formando um único vetor (vertical ou horizontal). Um exemplo está na figura “Operação de achatamento de uma imagem”.

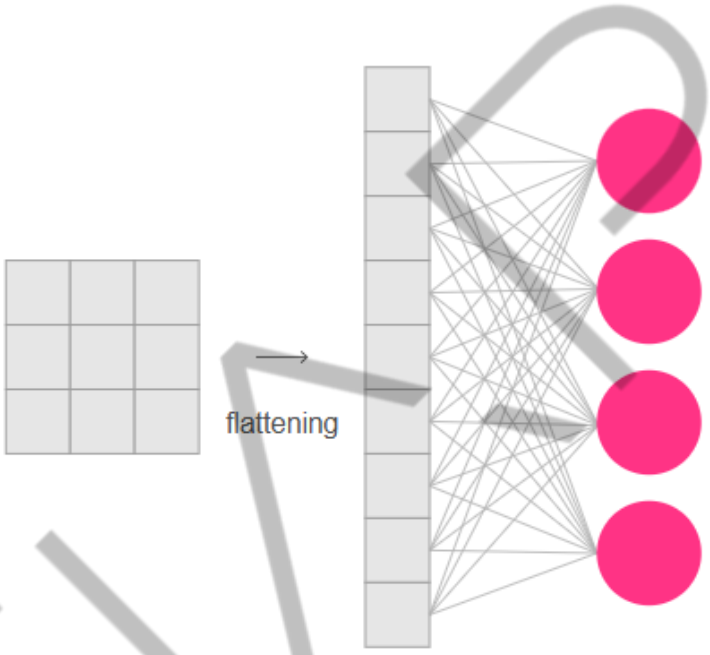

Figura 33 - Operação de achatamento de uma imagem
Fonte: Stackoverflow (2017), adaptado por FIAP (2025)

Podemos fazer este achatamento por meio do código a seguir, o qual também não gera saída:

```
# Torna as imagens lineares - Entrada para o Sklearn
def flatten(f):
  flat_list = [item for sublist in f for item in sublist]
  return flat_list

# Criar as features para uso do sklearn
train_images_features = [ flatten(image) for image in
train_images ]
test_images_features = [ flatten(image) for image in
test_images ]
```

Código-fonte 6 - Normalizando os dados
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora é a hora de definir o modelo e treiná-lo. Vamos começar com o método que já conhecemos: via Sklearn. O código a seguir define a arquitetura da rede neural, treina o modelo e calcula a acurácia:

```
# Arquitetura da rede
ann = MLPClassifier(hidden_layer_sizes=(128,),
                    activation="relu",
                    solver="adam",
                    learning_rate_init=0.001,
                    max_iter=10,
                    random_state=42,
                    verbose=True)

# Treinamento do modelo
ann.fit(train_images_features, train_labels)

# Predição e avaliação
preds = ann.predict(test_images_features)
print("Acurácia: ", accuracy_score(test_labels, preds))
```

Código-fonte 7 - Rede neural construída com Sklearn
Fonte: Elaborado pelo autor (2024)

Para este modelo é esperada uma acurácia de **0,8811**. Vamos então para o modelo definido com TensorFlow:

```
# Definição da semente de aleatoriedade
tf.random.set_seed(42)

# Define o modelo
model = keras.Sequential( [
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
]                                                          )

# Compilar o modelo
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"]
              )
```

Código-fonte 8 - Rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

O código-fonte anterior indica que a forma de utilização do TensorFlow é parecida com o Sklearn. A diferença é que este framework exige que a rede neural seja explicitamente definida, camada a camada. Ela inicia com uma camada de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

achatamento (dispensando o tratamento feito para o Sklearn), depois temos uma camada oculta densa com 128 neurônios e uma camada de decisão densa com 10 neurônios (um para cada classe do problema). Também utilizamos a função de ativação “relu” nas camadas ocultas.

Outra diferença é que o modelo definido no TensorFlow precisa ser compilado. Nesta etapa informamos o algoritmo de otimização de busca de pesos, a loss (função de perda) a ser empregada e qual métrica será otimizada durante o treinamento (no caso, acurácia).

Uma vantagem do TensorFlow é que é possível exibir a arquitetura da rede montada com apenas um comando:

```
# O que tem neste modelo?
model.summary()
```

Código-fonte 9 - Resumindo a rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

Gerando, assim, a saída da figura a seguir, onde indica que a rede possui três camadas e para cada uma delas qual o tamanho da entrada e o número de parâmetros (pesos e bias). Note que temos 107.770 parâmetros a serem aprendidos em tempo de treinamento.

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten_1 (Flatten)         (None, 784)               0

 dense_2 (Dense)             (None, 128)               100480

 dense_3 (Dense)             (None, 10)                1290

=================================================================
Total params: 101770 (397.54 KB)
Trainable params: 101770 (397.54 KB)
Non-trainable params: 0 (0.00 Byte)
```

Figura 34 - Saída da execução do Código-fonte 9
Fonte: Elaborado pelo autor (2024)

Chegou a hora de treinar o modelo! Para fazer da mesma forma que no Sklearn, precisamos utilizar o método “fit()”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Treinamento do modelo
model.fit(train_images, train_labels, epochs=10)
```

Código-fonte 10 - Treinando a rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

Gerando a saída da seguinte figura:

```
Epoch 1/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.4904 - accuracy: 0.8277
Epoch 2/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.3692 - accuracy: 0.8658
Epoch 3/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.3335 - accuracy: 0.8781
Epoch 4/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.3111 - accuracy: 0.8870
Epoch 5/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.2933 - accuracy: 0.8913
Epoch 6/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.2787 - accuracy: 0.8959
Epoch 7/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.2685 - accuracy: 0.9002
Epoch 8/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.2553 - accuracy: 0.9054
Epoch 9/10
1875/1875 [==============================] - 7s 3ms/step - loss: 0.2462 - accuracy: 0.9076
Epoch 10/10
1875/1875 [==============================] - 5s 3ms/step - loss: 0.2410 - accuracy: 0.9103
CPU times: user 1min 13s, sys: 5.09 s, total: 1min 18s
Wall time: 1min 3s
<keras.src.callbacks.History at 0x7a8fe421ace0>
```

Figura 35 - Saída da execução do Código-fonte 10
Fonte: Elaborado pelo autor (2024)

Para encerrar, precisamos validar o modelo construído, o qual gera uma acurácia de **0,8845** (similar ao resultado passado; pequenas alterações se devem à fatores de arredondamento).

```
# Avaliação do modelo
test_loss, test_acc = model.evaluate(test_images, test_labels)

print("Test loss: ", test_loss)
print("Test  ACC: ", test_acc)
```

Código-fonte 11 - Treinando a rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

Em resumo, nesta prática guiada aprendemos que:

- A interface de uso do TensorFlow é similar ao do Sklearn;
- No TensorFlow precisamos definir o modelo camada a camada e compilá-lo;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- O TensorFlow permite inspecionar a arquitetura construída, apontando quantos parâmetros precisam ser ajustados durante o treinamento;
- Para redes bem simples (como este caso) os resultados nos dois frameworks são essencialmente iguais.

7.2 CNN em Python

Agora que já conhecemos o TensorFlow e compreendemos as CNNs, está na hora de praticarmos sua implementação. Nesta atividade começaremos a criar redes neurais profundas, via TensorFlow. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 2b), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Dica: para esta parte do código, utilizaremos um dataset já salvo na memória do TensorFlow, por isso não há arquivos externos associados!

Utilizaremos a base “tf_flowers” já persistida no TensorFlow. Essa base contém imagens de cinco classes diferentes de flores: rosas, margaridas, dente de leão, girassóis e tulipas. Cada classe contém um número variável de imagens de alta resolução, capturadas de diferentes ângulos e condições de iluminação. Essa base de dados é frequentemente usada para tarefas de classificação de imagens, como treinamento e avaliação de modelos de CNNs para reconhecimento de flores. Exemplos de dados desta base estão na figura a seguir. Observe:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

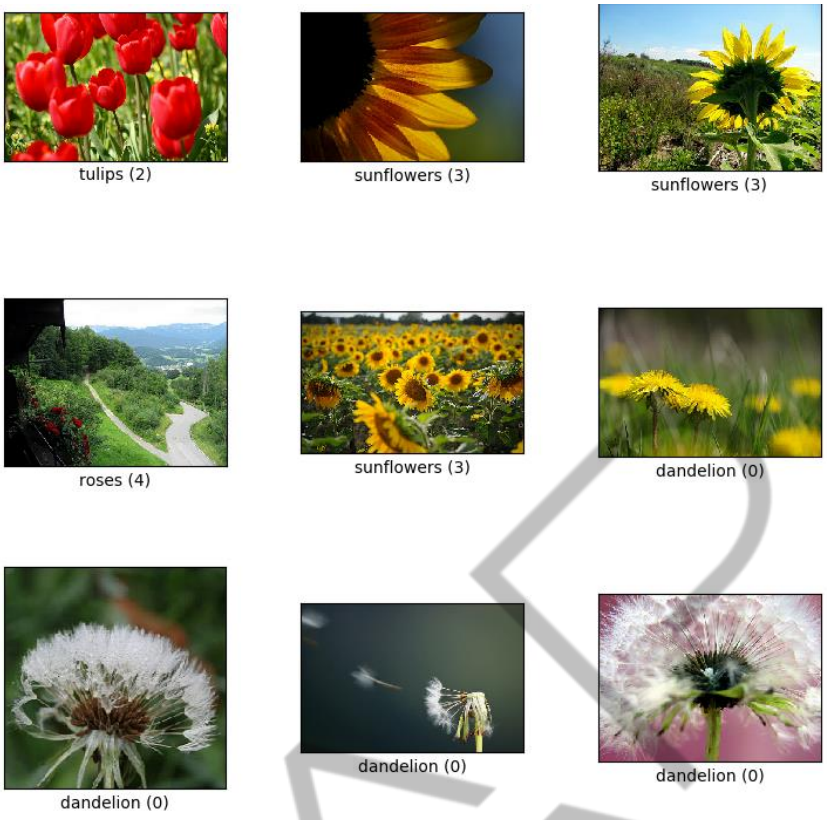

Figura 36 - Exemplos da base TF_Flowers
Fonte: TensorFlow (2023)

Vamos começar com os *imports*:

```
# Imports
import tensorflow as tf
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix

from keras.api._v2.keras import callbacks
from tensorflow.keras import Sequential, layers
from tensorflow.keras.callbacks import EarlyStopping
from  tensorflow.keras.layers.experimental.preprocessing
import Rescaling

from tensorflow.keras.applications.vgg16 import VGG16
from      tensorflow.keras.applications.vgg16      import
preprocess_input

from tensorflow.keras import layers, models
```

Código-fonte 12 - Imports do código de CNN
Fonte: Elaborado pelo autor (2024)

Este primeiro código não gera saída. Vamos carregar o dataset:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Carregar o dataset
import tensorflow_datasets as tfds
from tensorflow.keras.utils import to_categorical

(train_ds, train_labels), (test_ds, test_labels) =
tfds.load(
    "tf_flowers",
    split=["train[:70%]", "train[:30%]"],
    batch_size=-1,
    as_supervised=True
)
```

Código-fonte 13 - Carregando o dataset
Fonte: Elaborado pelo autor (2024)

O carregamento pode demorar alguns segundos, e, quando concluído, mostrará a barra de status em 100%. Note que já estamos definindo as variáveis que armazenarão o TREINO e o TESTE, bem como os LABELS. Caso você queira exibir o tamanho destes conjuntos, execute:

```
# Verificando o tamanho das variáveis
display("Tamanho do treino: ")
display(train_ds.shape)
display("Tamanho do treino: ")
display(test_ds.shape)
display("Labels do treino: ")
display(train_labels)
```

Código-fonte 14 - Chegando o tamanho dos conjuntos de treino e teste
Fonte: Elaborado pelo autor (2024)

Esse código mostra que o TREINO possui 2569 imagens, enquanto o TESTE possui 1101. No entanto, em ambos os conjuntos as imagens possuem 442x1024 pixels e são expressas em 3 canais (RGB). O tamanho do vetor de labels do treino também possui 2569 rótulos, que são números de 0 a 4 (representando as cinco classes do problema). Observe:

```
'Tamanho do treino: '
TensorShape([2569, 442, 1024, 3])
'Tamanho do treino: '
TensorShape([1101, 442, 1024, 3])
'Labels do treino: '
<tf.Tensor: shape=(2569,), dtype=int64, numpy=array([2, 3, 3, ..., 0, 2, 0])>
```

Figura 37 - Saída do Código-fonte 14
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora vamos reescalar as imagens para um tamanho menor, fazendo com que a rede treine mais rápido. Imagens de alta resolução demandam maior poder computacional. Queremos que as imagens fiquem com 150x150 pixels, mantendo o aspecto das cores. Também representaremos os labels de forma categórica. Assim, o label "`0`" vira o vetor "`[1., 0., 0., 0., 0.]`", o label "`1`" vira o vetor "`[0., 1., 0., 0., 0.]`", e assim sucessivamente:

```
# Tamanho alvo: 150x150 pixels
size = (150, 150)

# Redimensionando os conjuntos de treino e teste
train_ds = tf.image.resize(train_ds, size)
test_ds = tf.image.resize(test_ds, size)

# Aplicando a representação categórica dos labels
train_labels = to_categorical(train_labels,
num_classes=5)
test_labels = to_categorical(test_labels, num_classes=5)

# Verificando o tamanho das variáveis
display("Novas dimensões do treino: ")
display(train_ds.shape)
display("Nova representação dos labels: ")
display(train_labels)
```

Código-fonte 15 - Pré-processamento dos dados
Fonte: Elaborado pelo autor (2024)

A saída será a exibida na próxima figura, a qual confirma que as imagens foram reescaladas, e os labels foram recodificados, conforme solicitado:

```
'Novas dimensões do treino: '
TensorShape([2569, 150, 150, 3])
'Nova representação dos labels: '
array([[0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0.],
       ...,
       [1., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [1., 0., 0., 0., 0.]], dtype=float32)
```

Figura 38 - Saída do Código-fonte 15
Fonte: Elaborado pelo autor (2024)

Vamos criar o primeiro modelo "profundo". Ele não terá tantas camadas quanto os modelos modernos, mas exemplificará como encadear camadas convolucionais e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de pooling. A rede criada a mão será sequencial, tendo uma camada após a outra. Ela começa com o reescalamento dos pixels, dividindo-os por 255. Aqui já informamos que receberemos imagens 150x150x3.

Feito isso, adicionamos camada a camada, sempre combinando uma camada convolucional com um max pooling. Aqui não há regras: podemos encadear várias convoluções, fazer poolings seguidos... estamos definhando uma arquitetura! A rede termina com camadas totalmente conectadas para fazer a classificação.

```
     # Definir a arquitetura da CNN -> camada de extração de
features
     hand_model = Sequential()
     hand_model.add( Rescaling(1./255, input_shape=(150, 150,
3)) )

     hand_model.add(     layers.Conv2D(16,     kernel_size=10,
activation="relu") )
     hand_model.add( layers.MaxPooling2D(3) )

     hand_model.add(     layers.Conv2D(32,     kernel_size=8,
activation="relu") )
     hand_model.add( layers.MaxPooling2D(2) )

     hand_model.add(     layers.Conv2D(32,     kernel_size=6,
activation="relu") )
     hand_model.add( layers.MaxPooling2D(2) )

     # Definir a arquitetura da CNN -> camada de decisão
(classificação)
     hand_model.add( layers.Flatten() )                      #
entrada
     hand_model.add(    layers.Dense(50,    activation="relu")
)    # 1ª camada oculta
     hand_model.add(    layers.Dense(20,    activation="relu")
)    # 2ª camada oculta
     hand_model.add(   layers.Dense(5,   activation="softmax")
)  # camada de decisão

     # Compilar o modelo
     hand_model.compile(
         optimizer="adam",
         loss="categorical_crossentropy",
         metrics=["accuracy"]
     )

     # Definir a estratégia de Early Stopping
     es = EarlyStopping(
         monitor="val_accuracy",
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            patience=5,
            restore_best_weights=True
        )

        # Treina o modelo
        with tf.device("/device:GPU:0"):
          hand_model.fit(
              train_ds, train_labels,
              epochs=50,
              validation_split=0.2,
              batch_size=32,
              callbacks=[es]
          )
```

Código-fonte 16 - Treinamento do modelo baseline, criado a mão
Fonte: Elaborado pelo autor (2024)

Uma vez que a arquitetura está montada, temos que compilar o modelo e treiná-lo (utilizando GPU). Para agilizar, definiremos um *Early Stopping* se o treinamento não melhorar significativamente após cinco épocas.

A saída será a exibida na figura “Saída do Código-fonte 16”, indicando que o treinamento parou após 18 épocas:

```
Epoch 11/50
65/65 [==============================] - 1s 19ms/step - loss: 0.8763 - accuracy: 0.6433 - val_loss: 1.1988 - val_accuracy: 0.5350
Epoch 12/50
65/65 [==============================] - 1s 19ms/step - loss: 0.8669 - accuracy: 0.6574 - val_loss: 1.0833 - val_accuracy: 0.5720
Epoch 13/50
65/65 [==============================] - 1s 19ms/step - loss: 0.7719 - accuracy: 0.7041 - val_loss: 1.1179 - val_accuracy: 0.6051
Epoch 14/50
65/65 [==============================] - 1s 19ms/step - loss: 0.7255 - accuracy: 0.7178 - val_loss: 1.2479 - val_accuracy: 0.5875
Epoch 15/50
65/65 [==============================] - 1s 21ms/step - loss: 0.6484 - accuracy: 0.7431 - val_loss: 1.1360 - val_accuracy: 0.5700
Epoch 16/50
65/65 [==============================] - 1s 21ms/step - loss: 0.5974 - accuracy: 0.7655 - val_loss: 1.1533 - val_accuracy: 0.5720
Epoch 17/50
65/65 [==============================] - 1s 19ms/step - loss: 0.5474 - accuracy: 0.7922 - val_loss: 1.3480 - val_accuracy: 0.5875
Epoch 18/50
65/65 [==============================] - 1s 19ms/step - loss: 0.4855 - accuracy: 0.8185 - val_loss: 1.5315 - val_accuracy: 0.5350
```

Figura 39 - Saída do Código-fonte 16
Fonte: Elaborado pelo autor (2024)

Vamos para a avaliação! Como é uma tarefa de classificação, calcularemos a acurácia e a matriz de confusão:

```
        # Avaliação do modelo
        loss, acc = hand_model.evaluate(test_ds, test_labels)

        print(f"A loss do modelo é {loss:.2f} e a ACC é
{acc:.2f}")
```

Código-fonte 17 - Calculando a acurácia do modelo inicial
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A acurácia do modelo feito a mão é:

```
35/35 [==============================] - 1s 15ms/step - loss: 0.6375 - accuracy: 0.7629
A loss do modelo é 0.64 e a ACC é 0.76
```

Figura 40 - Saída do Código-fonte 17
Fonte: Elaborado pelo autor (2024)

Em termos de matriz de confusão, precisaremos do seguinte código:

```
# Função para obter a matriz de confusão
def plot_confusion_matrix(y_test, y_pred):
  labels = list( map( np.argmax, y_test ) )
  labels_pred = list( map( np.argmax, y_pred ) )

  cf_matrix = confusion_matrix(labels, labels_pred)
  sns.heatmap(cf_matrix, annot=True)

# Matriz de confusão
preds = hand_model.predict(test_ds)
plot_confusion_matrix(test_labels, preds)
```

Código-fonte 18 - Plotando a matriz de confusão do modelo inicial
Fonte: Elaborado pelo autor (2024)

Que gera a saída da figura a seguir:

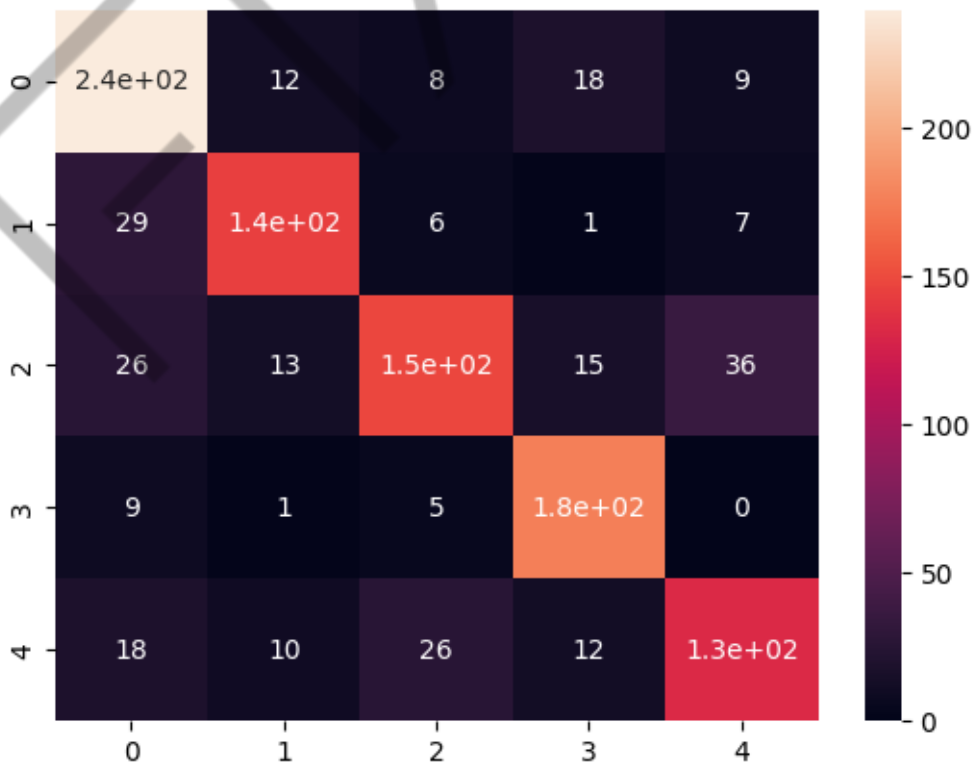

Figura 41 - Saída do Código-fonte 18
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para um modelo simples, é um bom resultado. Mas podemos fazer melhor! Vamos usar a técnica de Transfer Learning, utilizando um modelo de base poderoso para extrair boas features! Nosso transfer será realizado a partir do modelo VGG16.

Primeiramente, temos que reprocessar as imagens trazendo-as para a mesma distribuição dos dados que treinaram essa VGG, que, neste caso, foi o ImageNet. Note que no nosso import o método "preprocess_input()" foi importado da VGG16 do TensorFlow. Esse ajuste é extremamente importante sempre que formos realizar o transfer de outro modelo.

```
     # Preprocessa os dados de treinamento e teste
     train_dsTL = preprocess_input(train_ds)
     test_dsTL = preprocess_input(test_ds)

     # Importar o modelo de base, que será a VGG16 ***sem*** a
camada de decisão
     base_model = VGG16(weights="imagenet", include_top=False,
                        input_shape=train_dsTL[0].shape)

     # Usar os mesmos pesos da rede treinada (sem fazer fine
tuning)
     base_model.trainable = False
```

Código-fonte 19 - Pré-processamento dos dados para uso da VGG16 para Transfer Learning
Fonte: Elaborado pelo autor (2024)

O código também importa a VGG16 como modelo de base, usando os pesos treinados no ImageNet, mas sem incluir a camada "top", que são as camadas de classificação. Ao indicarmos que o modelo não é "treinável", estamos informando ao TensorFlow para usar os mesmos pesos da rede já pré-treinada, ou seja, use todas as camadas do modelo de base como um grande extrator de features.

A saída desta célula é apenas o "download" dos pesos pré-configurados. O mais interessante é observarmos a composição do modelo de base:

```
     # Observa a estrutura do modelo importado
     base_model.summary(
```

Código-fonte 20 - Observando as camadas da VGG16
Fonte: Elaborado pelo autor (2024)

A saída está na figura a seguir. Note quantas camadas existem no modelo VGG16.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Model: "vgg16"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 150, 150, 3)]     0

 block1_conv1 (Conv2D)       (None, 150, 150, 64)      1792

 block1_conv2 (Conv2D)       (None, 150, 150, 64)      36928

 block1_pool (MaxPooling2D)  (None, 75, 75, 64)        0

 block2_conv1 (Conv2D)       (None, 75, 75, 128)       73856

 block2_conv2 (Conv2D)       (None, 75, 75, 128)       147584

 block2_pool (MaxPooling2D)  (None, 37, 37, 128)       0

 block3_conv1 (Conv2D)       (None, 37, 37, 256)       295168

 block3_conv2 (Conv2D)       (None, 37, 37, 256)       590080

 block3_conv3 (Conv2D)       (None, 37, 37, 256)       590080

 block3_pool (MaxPooling2D)  (None, 18, 18, 256)       0

 block4_conv1 (Conv2D)       (None, 18, 18, 512)       1180160

 block4_conv2 (Conv2D)       (None, 18, 18, 512)       2359808

 block4_conv3 (Conv2D)       (None, 18, 18, 512)       2359808

 block4_pool (MaxPooling2D)  (None, 9, 9, 512)         0

 block5_conv1 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_conv2 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_conv3 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_pool (MaxPooling2D)  (None, 4, 4, 512)         0

=================================================================
Total params: 14714688 (56.13 MB)
Trainable params: 0 (0.00 Byte)
Non-trainable params: 14714688 (56.13 MB)
```

Figura 42 - Saída do Código-fonte 20
Fonte: Elaborado pelo autor (2024)

Observe também que este modelo possui mais de 14 milhões de parâmetros! No entanto, nenhuma dele é “treinável” pois já estão todos treinados. Usaremos estes +14 milhões de pesos já ajustados no ImageNet como extratores de features das imagens de flores! O que falta, então, é definirmos a camada de decisão (classificação):

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     # Define as camadas
     flatten_layer = layers.Flatten()
     dense_layer_1 = layers.Dense(50, activation="relu")
     dense_layer_2 = layers.Dense(20, activation="relu")
     prediction_layer = layers.Dense(5, activation="softmax")

     # Estabelece o modelo completo
     model = models.Sequential( [
         base_model,         # A base CNN vem da VGG16
          flatten_layer,      # Achata as features que saem da
VGG16 (base_model)
         dense_layer_1,      # 1ª camada oculta fully connected
         dense_layer_2,      # 2ª camada oculta fully connected
         prediction_layer    # Camada de saída, com 5 neurônios
para classificação
     ]                                                          )

     model.summary()
```

Código-fonte 21 - Completando o modelo de transfer com camadas de classificação
Fonte: Elaborado pelo autor (2024)

A saída está na figura “Saída do Código-fonte 21”. Note que o modelo completo possui o início da VGG e depois as camadas de decisão que adicionamos, exatamente iguais ao modelo definido a mão (baseline).

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 vgg16 (Functional)          (None, 4, 4, 512)         14714688

 flatten_1 (Flatten)         (None, 8192)              0

 dense_3 (Dense)             (None, 50)                409650

 dense_4 (Dense)             (None, 20)                1020

 dense_5 (Dense)             (None, 5)                 105

=================================================================
Total params: 15125463 (57.70 MB)
Trainable params: 410775 (1.57 MB)
Non-trainable params: 14714688 (56.13 MB)
```

Figura 43 - Saída do Código-fonte 21
Fonte: Elaborado pelo autor (2024)

Nesta saída fica mais evidente que a quantidade de parâmetros aumentou, sendo que continuamos com mais de 14 milhões de parâmetros já treinados, porém temos mais de 410 mil a serem aprendidos, referentes às camadas de decisão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A partir deste ponto, o processo de compilar e treinar o modelo é semelhante. Para uma comparação justa, também acionaremos o *Early Stopping* após cinco épocas sem alterações profundas na métrica de validação.

```
# Compilar o modelo
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Definir a estratégia de Early Stopping
es = EarlyStopping(
    monitor="val_accuracy",
    patience=5,
    restore_best_weights=True
)

# Treina o modelo
with tf.device("/device:GPU:0"):
  model.fit(
        train_dsTL, train_labels,  # alterei o nome do
conjunto de treino, usando o train_dsTL
      epochs=50,
      validation_split=0.2,
      batch_size=32,
      callbacks=[es]
  )
```

Código-fonte 22 - Compilando o treinando o modelo resultante
Fonte: Elaborado pelo autor (2024)

A saída é o histórico de métricas em tempo de treinamento. Vamos direto para a avaliação, também em termos de acurácia e matriz de confusão:

```
# Avaliação do modelo: acurácia
loss, acc = model.evaluate(test_dsTL, test_labels)

print(f"A loss do modelo é {loss:.2f} e a ACC é
{acc:.2f}")

# Avaliação do modelo: matriz de confusão
preds = model.predict(test_dsTL)
plot_confusion_matrix(test_labels, preds)
```

Código-fonte 23 - Avaliação do modelo com Transfer Learning
Fonte: Elaborado pelo autor (2024)

A saída esperada está na seguinte figura. Observe:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
35/35 [==============================] - 2s 65ms/step - loss: 0.1232 - accuracy: 0.9628
A loss do modelo é 0.12 e a ACC é 0.96
35/35 [==============================] - 2s 61ms/step
```

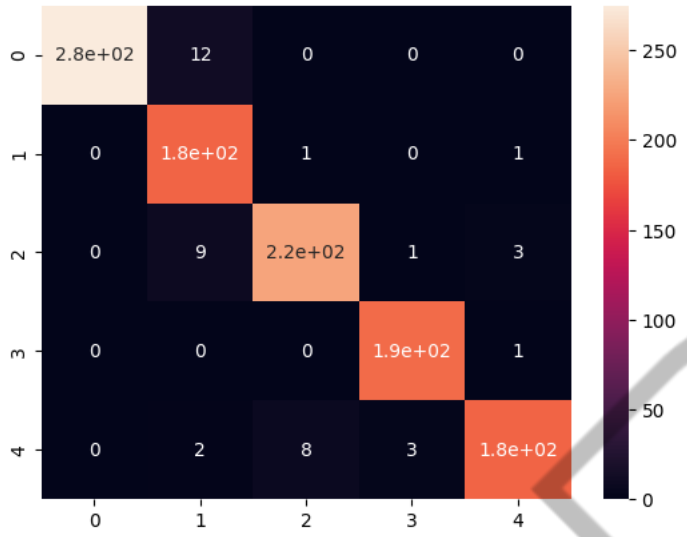

Figura 44 - Saída do Código-fonte 23
Fonte: Elaborado pelo autor (2024)

Muito melhor, não é mesmo? Nossa acurácia subiu para 96% e quase não há confusão entre as classes!

Este código nos mostrou que o Transfer Learning é um artifício valioso para gerar modelos profundos sem precisar treinar do zero as redes, mas aproveitando modelos pré-treinados como bons extratores genéricos de features, as quais podem ser “aprendidas” por outros algoritmos de Machine Learning, incluindo redes neurais totalmente conectadas.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ADALOGLOU, Nikolas. Best deep CNN architectures and their principles: from AlexNet to EfficientNet. **The AI Summer**, 21 de janeiro de 2021. Disponível em: <https://theaisummer.com/cnn-architectures/>. Acesso em: 12 fev. 2025.

BAKKER, Indra den. Battle of the Deep Learning frameworks — Part I: 2017, even more frameworks and interfaces. **Medium**, 19 de dezembro de 2017. Disponível em: <https://medium.com/towards-data-science/battle-of-the-deep-learning-frameworks-part-i-cff0e3841750>. Acesso em: 12 fev. 2025.

BISSOTO, Alceu, *et al*. **Skin lesion synthesis with generative adversarial networks**. In: OR 2.0 Context-Aware Operating Theaters, Computer Assisted Robotic Endoscopy, Clinical Image-Based Procedures, and Skin Image Analysis: First International Workshop, OR 2.0 2018, 5th International Workshop, CARE 2018, 7th International Workshop, CLIP 2018, Third International Workshop, ISIC 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16 and 20, 2018, Proceedings 5. Springer International Publishing, 2018. p. 294-302.

BISSOTO, Alceu; PEREZ, Fábio; VALLE, Eduardo; AVILA, Sandra. Skin Lesion Synthesis with Generative Adversarial Networks. **arXiv**. Disponível em: <https://arxiv.org/pdf/1902.03253>. Acesso em: 12 fev. 2025.

CALIN, Ovidiu. **Deep learning architectures**. New York City: Springer International Publishing, 2020.

CAPÍTULO 54 – Introdução às Redes Adversárias Generativas (GANs – Generative Adversarial Networks). **Deep Learning Book**, 2022. Disponível em: <https://www.deeplearningbook.com.br/introducao-as-redes-adversarias-generativas-gans-generative-adversarial-networks/>. Acesso em: 12 fev. 2025.

CHOWDARY, M. Kalpana; ANITHA, J.; HEMANTH, D. Jude. Emotion Recognition from EEG Signals Using Recurrent Neural Networks. **MDPI**, 30 de julho de 2022. Disponível em: <https://www.mdpi.com/2079-9292/11/15/2387>. Acesso em: 12 fev. 2025.

CNN | Introduction to Pooling Layer. **GeeksforGeeks**, 4 de fevereiro de 2025. Disponível em: <https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/>. Acesso em: 12 fev. 2025.

CONVOLUTIONAL Neural Networks: 1998-2023 Overview. **SuperAnnotate**, 15 maio de 2023. Disponível em: <https://www.superannotate.com/blog/guide-to-convolutional-neural-networks>. Acesso em: 14 fev. 2025.

COOPER, Gordon. New Vision Technologies For Real-World Applications. **Semiconductor Engineering**, 03 de outubro de 2019. Disponível em: <https://semiengineering.com/new-vision-technologies-for-real-world-applications/>. Acesso em: 12 fev. 2025.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

DESHPANDE, Adit. A Beginner's Guide To Understanding Convolutional Neural Networks. **GitHub**, 20 de julho de 2016. Disponível em: <https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/>. Acesso em: 12 fev. 2025.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep learning**. MIT press, 2016.

GUILLOU, Pierre. Qual é o princípio de funcionamento de um algoritmo de inteligência artificial? **Medium**, 02 de maio de 2018. Disponível em: <https://medium.com/@pierre_guillou/qual-%C3%A9-o-principio-de-funcionamento-de-um-algoritmo-de-intelig%C3%AAncia-artificial-d68619ce2b4>. Acesso em: 12 fev. 2025.

GUPTA, Sanchit. A simple solution to change pixel values in an excel file to .jpg using Python. **Medium**, 02 de dezembro de 2020. Disponível em: <https://medium.com/lifeandtech/convert-csv-file-to-images-309b6fdb8c49>. Acesso em: 12 fev. 2025.

HAN, Xiaobing; ZHONG, Yanfei; CAO, Liqin; ZHANG, Liangpei. Pre-Trained AlexNet Architecture with Pyramid Pooling and Supervision for High Spatial Resolution Remote Sensing Image Scene Classification. **MDPI**, 16 de agosto de 2017. Disponível em: <https://www.mdpi.com/2072-4292/9/8/848>. Acesso em: 12 fev. 2025.

JENIK, Claire. A Minute on the Internet in 2021. **Statista**, 30 de julho de 2021. Disponível em: <https://www.statista.com/chart/25443/estimated-amount-of-data-created-on-the-internet-in-one-minute/>. Acesso em: 12 fev. 2025.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. **Imagenet classification with deep convolutional neural networks**. Advances in neural information processing systems, 2012.

LUSOV, Kyrylo. Structured vs Unstructured Data – What is the Difference? **Jelvix**, 2024. Disponível em: <https://jelvix.com/blog/structured-vs-unstructured-data>. Acesso em: 12 fev. 2025.

MISHRA, Ranjan Kumar; REDDY, GY Sandesh; PATHAK, Himanshu. **The understanding of deep learning:** A comprehensive review. Mathematical Problems in Engineering, 2021, 2021: 1-15.

MOTA, Matheus. Diferença de IA, ML e DL: Desvendando as Diferenças Fundamentais entre IA, ML e DL. **Medium**, 28 de agosto de 2020. Disponível em: <https://medium.com/@matheus1714/diferen%C3%A7a-de-ia-ml-e-dl-6f1a06882859>. Acesso em: 12 fev. 2025.

PRIJONO, Benny. Student Notes: Convolutional Neural Networks (CNN) Introduction. **IndoML**, 07 de março de 2018. Disponível em: <https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/>. Acesso em: 12 fev. 2025.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

ROMAN, Victor. Convolutional Neural Networks. **Towards Data Science**, 27 de fevereiro de 2020. Disponível em: <https://towardsdatascience.com/convolutional-neural-networks-357b9b2d75bd/>. Acesso em: 14 fev. 2025.

SHARMA, Mathanraj. Understanding Images with skimage-Python. **Medium**, 31 de outubro de 2019. Disponível em: <https://medium.com/towards-data-science/understanding-images-with-skimage-python-b94d210afd23>. Acesso em: 12 fev. 2025.

TAN, Mingxing; LE, Quoc. **Efficientnet:** Rethinking model scaling for convolutional neural networks. In: International conference on machine learning. PMLR, 2019. p. 6105-6114.

TF_FLOWERS. **TensorFlow**, 11 de fevereiro de 2023. Disponível em: <https://www.tensorflow.org/datasets/catalog/tf_flowers?hl=pt-br>. Acesso em: 12 fev. 2025.

VALLE, Eduardo, *et al*. **Data, depth, and design:** Learning reliable models for skin lesion analysis. Neurocomputing, 2020, 383: 303-313.

VASILEV, Ivan, *et al*. **Python Deep Learning:** Exploring deep learning techniques and neural network architectures with Pytorch, Keras, and TensorFlow. Packt Publishing Ltd, 2019.

VEEN, Fjodor van. Neural Network Zoo Prequel: Cells and Layers**. The Asimov Institute**, 31 de março de 2017. Disponível em: <https://www.asimovinstitute.org/author/fjodorvanveen/>. Acesso em: 12 fev. 2025.

WEI, Donglai; ZHOU, Bolei; TORRALBA, Antonio; FREEMAN, William T. mNeuron: A Matlab Plugin to Visualize Neurons from Deep Models. **GitHub**, [2020]. Disponível em: <https://donglaiw.github.io/proj/mneuron/index.html>. Acesso em: 12 fev. 2025.

WHAT deep learning is and isn't. **The Data Scientist**, 2018. Disponível em: <https://thedatascientist.com/what-deep-learning-is-and-isnt/>. Acesso em: 12 fev. 2025.

WHAT is the role of "Flatten" in Keras? **Stackoverflow**, 2017. Disponível em: https://stackoverflow.com/questions/43237124/what-is-the-role-of-flatten-in-keras. Acesso em: 14 fev. 2025.

고민수. Deep Learning Bible - 3. Natural Language Processing - Eng. **WikiDocs**, 16 de janeiro de 2025. Disponível em: <https://wikidocs.net/166316>. Acesso em: 12 fev. 2025.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
CNN	Convolutional Neural Network (Redes Neurais Convolucionais)
CPU	Central Processing Unit (Unidade de Processamento Central)
DL	Deep Learning
ELU	Exponential Linear Unit (Unidade Linear Exponencial)
FC	Fully Connected (Totalmente conectadas)
GANs	Generative Adversarial Network (Redes Adversárias Generativas)
GenAI	Generative AI (Inteligência Artificial Generativa)
GPU	Graphics Processing Unit (Unidade de Processamento Gráfico)
GRUs	Gated Recurrent Units (Unidades Recorrentes Fechadas)
IA	Inteligência Artificial
ILSVRC	ImageNet Large Scale Visual Recognition Challenge
IoT	Internet of Things
LSTMs	Long Short-Term Memory (Memória de Curto Longo Prazo)
ML	Machine Learning
PLN	Processamento de Linguagem Natural
ReLU	Rectified Linear Unit (Unidade Linear Retificada)
RGB	Red, Green, Blue (Vermelho, Verde, Azul)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com